

# Desarrollo de modelo para identificar cambios en soportes de cañerías sometidas a vibraciones

Ing. Carlos G. Massobrio

**Carrera de Especialización en Inteligencia Artificial**

**Director:** Dr. Ing. Marcos Maillot (UTN FRGP)

**Codirector:** Dr. Ing. Benjamín Tourn (UNRaf)

**Jurados:**

Dr. Lic. Tobias Canavesi (FIUBA)

MSc. Ing. Nahuel Pelli (FIUBA)

Mg. Ing. Pablo Slavkin (FIUBA)



## *Resumen*

Esta memoria presenta el uso de redes neuronales informadas por física, también conocidas por su sigla en inglés como PINN (*Physics-Informed Neural Networks*), para estudiar cañerías sometidas a vibraciones mecánicas. Se busca identificar cambios en las condiciones de diseño de los apoyos, poniendo a prueba diferentes configuraciones del modelo físico.

Los conocimientos adquiridos en el aprendizaje profundo han resultado de suma importancia para encarar el estudio de este nuevo campo de la inteligencia artificial.



# Índice general

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Resumen</b>                                                 | <b>I</b>  |
| <b>1. Introducción general</b>                                 | <b>1</b>  |
| 1.1. La inteligencia artificial para uso científico . . . . .  | 1         |
| 1.2. Motivación . . . . .                                      | 2         |
| 1.3. Redes neuronales informadas por física . . . . .          | 2         |
| 1.4. Estado del arte . . . . .                                 | 4         |
| 1.5. Objetivo y alcance . . . . .                              | 5         |
| <b>2. Introducción específica</b>                              | <b>7</b>  |
| 2.1. Redes neuronales de avance . . . . .                      | 7         |
| 2.2. Entrenamiento de un red neuronal . . . . .                | 8         |
| 2.2.1. Función de pérdida . . . . .                            | 8         |
| 2.2.2. Algoritmo de <i>backpropagation</i> . . . . .           | 8         |
| 2.2.3. Optimización . . . . .                                  | 8         |
| 2.3. Modelado de PINN . . . . .                                | 9         |
| 2.3.1. Puntos de colocación . . . . .                          | 9         |
| 2.3.2. Función de pérdida . . . . .                            | 10        |
| 2.3.3. Esquema principal . . . . .                             | 10        |
| <b>3. Diseño e implementación</b>                              | <b>13</b> |
| 3.1. Desarrollo de los modelos físicos . . . . .               | 13        |
| 3.1.1. Modelo físico específico . . . . .                      | 14        |
| 3.1.2. Modelo físico general . . . . .                         | 16        |
| 3.2. Consideraciones generales de los modelos PINN . . . . .   | 18        |
| 3.2.1. Características del <i>pipeline</i> . . . . .           | 18        |
| 3.2.2. Conjuntos de datos . . . . .                            | 19        |
| 3.2.3. Configuración de la red FNN . . . . .                   | 20        |
| 3.2.4. Aspectos específicos de los modelos PINN . . . . .      | 21        |
| 3.3. Características del código . . . . .                      | 22        |
| 3.3.1. Implementaciones del <i>framework</i> Pytorch . . . . . | 22        |
| 3.3.2. Estructura . . . . .                                    | 24        |
| 3.4. Gestión de ensayos . . . . .                              | 26        |
| <b>4. Ensayos y resultados</b>                                 | <b>27</b> |
| 4.1. Presentación general . . . . .                            | 27        |
| 4.1.1. Recursos técnicos . . . . .                             | 28        |
| 4.1.2. Configuración de los ensayos . . . . .                  | 28        |
| 4.1.3. Planificación según los modelos físicos . . . . .       | 30        |
| 4.2. Ensayos del modelo específico . . . . .                   | 31        |
| 4.2.1. Problema directo . . . . .                              | 31        |
| 4.2.2. Problema inverso . . . . .                              | 33        |
| 4.3. Ensayos del modelo general . . . . .                      | 36        |

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>5. Conclusiones</b>                               | <b>39</b> |
| 5.1. Conclusiones generales . . . . .                | 39        |
| 5.2. Próximos pasos . . . . .                        | 40        |
| <b>A. Obtención de los parámetros adimensionales</b> | <b>41</b> |
| A.1. Modelo físico específico . . . . .              | 41        |
| A.2. Modelo físico general . . . . .                 | 42        |
| A.2.1. Definiciones . . . . .                        | 42        |
| A.2.2. Cálculo de valores . . . . .                  | 43        |
| <b>Bibliografía</b>                                  | <b>45</b> |

# Índice de figuras

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| 1.1. AI para ciencia y su desarrollo en mecánica computacional. . . . .         | 4  |
| 2.1. Esquema de una PINN para casos de vibraciones mecánicas. . . . .           | 11 |
| 3.1. Esquema del modelo físico específico. . . . .                              | 15 |
| 3.2. Esquema del modelo físico general. . . . .                                 | 16 |
| 4.1. Pérdidas para problema directo del modelo específico . . . . .             | 32 |
| 4.2. Inferencias para el modelo directo del modelo específico . . . . .         | 33 |
| 4.3. Puntos de colocación para problema inverso del modelo específico . . . . . | 34 |
| 4.4. Inferencias para el modelo inverso del modelo específico . . . . .         | 35 |





# Índice de tablas

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| 4.1. Conjunto de datos para problema directo del modelo específico . .       | 31 |
| 4.2. Hiperparámetros para el problema directo del modelo específico .        | 32 |
| 4.3. Error relativo $L^2$ para el problema directo del modelo específico . . | 33 |
| 4.4. Conjunto de datos para problema inverso del modelo específico . .       | 34 |
| 4.5. Hiperparámetros para el problema inverso del modelo específico .        | 34 |
| 4.6. Resultados para problema inverso del modelo específico . . . . .        | 35 |
| 4.7. Referencia de cantidades de puntos de colocación del modelo general     | 36 |
| 4.8. Hiperparámetros del modelo general . . . . .                            | 36 |



# Capítulo 1

## Introducción general

Este capítulo proporciona una breve descripción general del contexto en el que se sitúa este trabajo en el ámbito científico-tecnológico. Se presenta de manera concisa la noción de redes neuronales informadas por física y su capacidad para integrar de manera efectiva los principios físicos fundamentales. Se realiza una revisión no exhaustiva del estado del arte en general y del empleo de estas técnicas en el análisis de vibraciones mecánicas y, finalmente, se establecen los objetivos y alcances del trabajo.

### 1.1. La inteligencia artificial para uso científico

En la actualidad, el auge de la integración de la inteligencia artificial (AI, *Artificial Intelligence* por su sigla en inglés) en el ámbito científico-tecnológico ha derivado en el concepto de AI para la ciencia. En este sentido, el uso de algoritmos de AI para resolver sistemas de ecuaciones en derivadas parciales constituye un asunto relevante en el ámbito de la mecánica computacional [1].

En términos generales, AI se identifica principalmente con el aprendizaje automático (conocido en inglés como *Machine Learning*). Incluye un tipo de modelos de aprendizaje computacional denominado aprendizaje profundo (conocido en inglés como *Deep Learnig*) [2], que se ha utilizado ampliamente en múltiples avances tecnológicos, consolidándose como una herramienta revolucionaria. Sus aplicaciones abarcan campos como la visión por computadora [3], el reconocimiento de voz [4], el procesamiento del lenguaje natural [5], los juegos de estrategia [6][7] y el desarrollo de fármacos [8]. Esta expansión ha tenido repercusiones directas en el ámbito científico-tecnológico, que ha incorporado activamente los desarrollos basados en aprendizaje profundo [9].

En particular, dado que numerosos fenómenos abordados en ciencia e ingeniería están formulados como ecuaciones en derivadas ordinarias (ODP), ecuaciones en derivadas parciales (EDP) o sistemas de ellos, el surgimiento estrategias basadas en herramientas de AI para su resolución [10] ha generado un creciente interés entre investigadores de física computacional y matemáticas aplicadas [11].

En el ámbito de la mecánica computacional, el uso del aprendizaje profundo para la resolución de EDPs emerge como referencia para el desarrollo de futuros enfoques capaces de acelerar significativamente los métodos numéricos convencionales. Recientemente a estos modelos se los ha denominado AI para EDP [1].

## 1.2. Motivación

Las cañerías que se encuentran sometidas a vibraciones mecánicas requieren el monitoreo de los esfuerzos dinámicos. El modelo matemático que describe al fenómeno físico consiste en un sistema EDP. Actualmente, entre las técnicas computacionales más utilizadas para resolverlas destacan los métodos de elementos finitos (MEF) [12] y los de volúmenes finitos (MVF) [13]. Estas técnicas de simulación se encuentran completamente desarrolladas y comprendidas. Sin embargo, una de las principales desventajas que presentan es que los mallados deben ser lo suficientemente densos como para capturar ciertos fenómenos locales, que inciden directamente en la solución del problema. En consecuencia, el costo computacional asociado suele ser muy elevado, lo que también implica prolongados tiempos de cálculo. Además, la modificación de algún parámetro del modelo físico (como la densidad del material o el módulo de elasticidad) implica resolver nuevamente el problema.

Las redes neuronales informadas por física, PINN por su siglas en inglés (*Physics-Informed Neural Networks*), son un tipo de enfoque de aprendizaje profundo identificable como modelo de AI para EDP [10]. Su diseño permite resolver fenómenos gobernados por principios físicos esenciales, porque restringen las predicciones a soluciones coherentes a tales principios y así no necesitan de muestras asociadas a valores objetivo; mientras que en los modelos convencionales de aprendizaje profundo, la red neuronal artificial se entrena<sup>1</sup> exclusivamente con conjuntos de muestras asociadas a valores objetivo.

El enfoque descrito es completamente novedoso en el uso de redes neuronales artificiales para resolver esta clase de problemas. Su reciente introducción en la literatura especializada abre múltiples líneas de investigación [1], relacionadas con detalles de la arquitectura, implementaciones, consideraciones específicas respecto a los problemas físicos a abordar, posibles acoplamientos con soluciones numéricas, entre otros aspectos. Esta motivación dio origen al proyecto orientado al estudio preliminar de PINN aplicado a vibraciones mecánicas. Con él se busca contribuir al desarrollo de herramientas que optimicen la ejecución de tareas de monitoreo de aptitud funcional de equipamiento industrial.

## 1.3. Redes neuronales informadas por física

PINN consiste fundamentalmente en imponer condiciones al proceso de entrenamiento de una red neuronal haciendo uso de las ecuaciones que definen un determinado fenómeno físico. La integración de conocimiento del campo de la física o las matemáticas en los modelos de aprendizaje automático es motivo de estudio en la comunidad científica en los últimos años. Por ejemplo, Lauer y coautores [14] incorporaron el conocimiento previo de las funciones y sus derivadas como restricciones en la regresión de vectores de soporte, conocido como SVR por su sigla en inglés (*Support Vector Regression*), para reducir los errores de aproximación.

Dos aspectos fundamentales merecen destacarse. En primer lugar, PINN es un método de colocación en el que permite aproximar la solución de la ecuación

---

<sup>1</sup>Se entiende por entrenamiento al proceso por el cual los algoritmos de aprendizaje automático adquieren experiencia para ejecutar tareas de forma autónoma.

diferencial mediante una red neuronal en base a un conjunto de puntos denominados `puntos de colocación`. A diferencia de MEF o MVF, PINN no requiere de un mallado. Estas particularidades hacen que los procedimientos para hallar la solución sean significativamente diferentes entre PINN y MEF. Por último, PINN aborda un problema de optimización en el que la función de pérdida presenta no linealidades y múltiples características particulares como puntos silla, mínimos locales o regiones con gradientes abruptos, mientras que el MEF resuelve un sistema de ecuaciones algebraicas linealizado localmente.

El segundo aspecto relevante de PINN es la construcción de la función de pérdida para entrenar la red neuronal a partir de los residuos de las ecuaciones que modelan matemáticamente los principios físicos. Esto restringe el espacio de búsqueda de soluciones y permite obtener una aproximación coherente con las leyes que gobiernan el sistema. Una ventaja de este enfoque es que, para calcular las derivadas presentes en las ecuaciones, se utilizan los mismos algoritmos de diferenciación automática empleados en las etapas de optimización durante el entrenamiento de la red.

Las características relevantes descritas de PINN permiten que posea las siguientes capacidades distintivas:

- Resolver el problema directo, es decir, calcular las variables dependientes de ecuaciones diferenciales sin necesidad de emplear un conjunto de datos rotulados (su uso es optativo). En este caso, se asume que los parámetros del modelo físico, las condiciones de borde y las condiciones iniciales son conocidos.
- Resolver el problema inverso. Para ello, solo se requiere incluir los coeficientes y/o funciones desconocidos del modelo físico como elementos a entrenar en la red neuronal. En este caso, sí resulta recomendable utilizar datos rotulados que pertenezcan al dominio de las ecuaciones diferenciales. Estos pueden generarse a partir de mediciones experimentales o simulaciones numéricas.
- Entrenar la red neuronal de modo que, además de las variables independientes, incluya como entrada un rango de valores correspondientes a los coeficientes del modelo físico. Esta estrategia permite realizar inferencias bajo diferentes configuraciones, lo que posibilita determinar el estado del sistema en estudio.

A modo de conclusión, puede decirse que PINN aprovecha las leyes físicas durante el entrenamiento, lo que reduce significativamente la necesidad de utilizar datos rotulados. Además, ofrece una mayor interpretabilidad en comparación con los enfoques basados exclusivamente en modelos de caja negra. Por otra parte, permite abordar problemas inversos y paramétricos mediante ajustes mínimos en su configuración. Sin embargo, la estructura no convexa de la función de pérdida, propia de las redes neuronales, introduce desafíos en el proceso de optimización. Aunque esta no convexidad aporta una notable capacidad de ajuste, también complica la convergencia durante el entrenamiento.

## 1.4. Estado del arte

La utilización de algoritmos de aprendizaje profundo para resolver EDPs comenzó en 2019 con la presentación de PINN de Raissi y coautores [10]. Sin embargo, el concepto de usar redes neuronales para resolver ecuaciones diferenciales se remonta a años previos [15][16][17]. Estas ideas no recibieron suficiente atención en su momento, ya que las tecnologías necesarias para ejecutar el cómputo requerido por las redes neuronales, como los procesadores para cálculo en paralelo y las herramientas de software asociadas, aún no estaban lo suficientemente desarrolladas o eran inexistentes.

Sin embargo, los avances de la última década en el campo de la AI habilitó el desarrollo de técnicas como PINN. Entre ellos se destacan el hardware optimizado para entrenar modelos de aprendizaje profundo y la implementación de algoritmos de diferenciación automática y de optimización en frameworks ampliamente utilizados, como PyTorch [18] y TensorFlow [19]. Estos avances han contribuido a que PINN gane terreno como herramienta válida en el ámbito de la física computacional y matemáticas. La figura 1.1 muestra el estado actual de la AI para ciencia. En el marco de la mecánica computacional, se mencionan métodos de resolución de sistemas EDP y casos de estudio en diversas disciplinas [1].

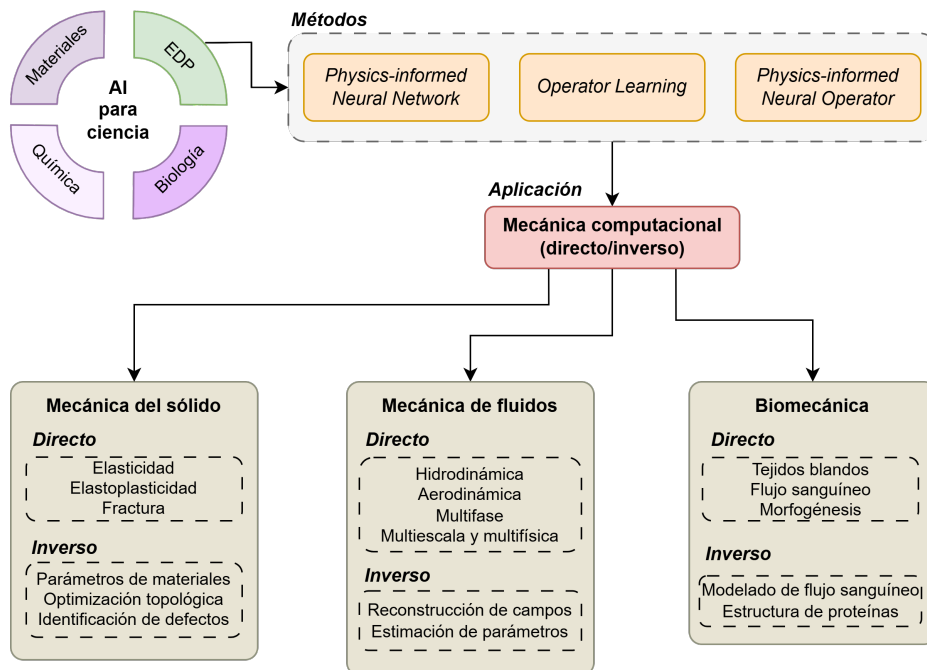


FIGURA 1.1. AI para ciencia y su desarrollo en mecánica computacional.

Por el momento, ha habido un limitado desarrollo de soluciones a problemas de estructuras sometidas a vibraciones mecánicas. Esta limitación se encuentra asociada con la dificultad de los modelos PINN para capturar aspectos relevantes en fenómenos físicos caracterizados por altas frecuencias. [20]. En este sentido, las investigaciones realizadas hasta el momento han intentado sortear las dificultades que presentan la resolución de sistemas dinámicos en estructuras mecánicas.

Diversos estudios recientes han abordado esta problemática. Entre estos se destaca el trabajo realizado por Yan y coautores [21], en el que se analiza el comportamiento elástico lineal de estructuras de placas. Le siguen los aportes realizados por Fallah y Aghdam [22] al analizar una viga de geometría variable sometida a vibraciones y restringida por una base elástica. Kapoor y coautores [23] también han hecho su aporte en el estudio de vigas sometidas a vibraciones mecánicas, evaluando distintos modelos físicos y configuraciones de restricciones. Por último, en un reciente trabajo de Söyleyici y Ünver [24], se expone una metodología para salvar los inconvenientes que se presentan al intentar estudiar el comportamiento de vigas sometidas a vibraciones mecánicas de alta frecuencia por medio de modelos PINN.

## 1.5. Objetivo y alcance

El proyecto pretende desarrollar un modelo PINN que cuente con la capacidad para resolver los siguientes problemas:

- Problema directo: computar las variables dependientes del modelo físico (por ejemplo, desplazamientos transversales) de un elemento estructural lineal sometido a vibraciones mecánicas.
- Problema inverso: obtener parámetros desconocidos del modelo físico previamente descrito a partir de conocer datos de las variables dependientes en algunas regiones del dominio.

La posibilidad de determinar el valor de parámetros de modelos físicos mediante la resolución del problema inverso resulta relevante para el monitoreo del estado del equipamiento industrial. Dado que las cañerías pueden considerarse como elementos estructurales, se optó por representarlas mediante un modelo de viga unidimensional del tipo Timoshenko [25]. Esta decisión responde a las complejidades asociadas al desarrollo del modelo físico completo, que podrían comprometer la viabilidad del proyecto. En este contexto, se establecieron las siguientes condiciones para la construcción de los prototipos, con especial atención a su aplicabilidad en entornos industriales:

- Modelizado de soportes considerando variaciones en las elasticidades lineales y rotacionales.
- Inclusión de una masa puntual entre los soportes de la viga.

Este trabajo propone explorar una nueva tecnología con alto potencial de impacto en la industria. En particular, en aplicaciones dedicadas a la evaluación de la integridad física y aptitud operacional de sistemas y equipos industriales.





## Capítulo 2

# Introducción específica

Este capítulo presenta los elementos fundamentales que sustentan el modelo PINN empleado en este trabajo. Primero, se profundiza en la estructura y funcionamiento de las redes neuronales. Luego, se detallan los elementos necesarios para completar el proceso de aprendizaje. Por último, se presentan los fundamentos de los modelos PINN y las particularidades a tener en cuenta para el análisis de vibraciones mecánicas.

### 2.1. Redes neuronales de avance

Las redes neuronales de avance, FNN por su sigla en inglés (*Feedforward Neural Networks*), tienen por finalidad aproximar cualquier función  $y$ . Desde una perspectiva matemática, una FNN realiza un mapeo  $\hat{y} = f(\mathbf{x}; \theta)$  y ajusta el valor de los parámetros  $\theta$  hasta encontrar la función que mejor se ajuste al valor deseado [2]. Se dice que las redes neuronales son de avance porque la información fluye en una única dirección a través de la función que se evalúa [2].

Por definición, dado un conjunto de datos  $\mathbf{x} \in \mathbb{D}$ , una FNN transforma dicha entrada en una salida a través de múltiples capas de unidades elementales (neuronas). Estas están compuestas por funciones que realizan mapeos lineales entre neuronas de capas sucesivas, así como por funciones de activación escalares no lineales aplicadas dentro de cada neurona, lo que da lugar a la siguiente representación [26] dada por 2.1:

$$\hat{y}(\mathbf{x}; \theta) = \mathbf{z}^{(K)} \circ \sigma \circ \mathbf{z}^{(K-1)} \dots \circ \sigma \circ \mathbf{z}^{(2)} \circ \sigma \circ \mathbf{z}^{(1)}(\mathbf{x}; \theta) \quad (2.1)$$

donde el símbolo  $\circ$  denota la composición sucesiva de funciones, combinando las funciones de transferencia y de activación desde la primera hasta la última capa  $K$ . El conjunto de parámetros de cada capa, que se ajustan durante el proceso de entrenamiento, están representados por  $\theta$  [26]. A partir de esta definición, se dice que la red se encuentra completamente conectada. También se dice que es, por su denominación en inglés, del tipo *fully connected*.

El desarrollo de la ecuación 2.1, esto es, la propagación sucesiva de una capa hacia la siguiente, desde la capa de entrada hasta la de salida, representa el proceso predictivo de la red neuronal. A esta operación se la conoce por su denominación en inglés como *forward propagation* [2].

## 2.2. Entrenamiento de un red neuronal

El entrenamiento de una FNN es un proceso iterativo cuyo objetivo es ajustar sus parámetros  $\theta$ , con el fin de minimizar una función de pérdida predefinida. Este procedimiento busca mejorar progresivamente la capacidad del modelo para realizar predicciones precisas. Para llevar a cabo este ajuste, es fundamental contar con un algoritmo de optimización que guíe la actualización de los parámetros en cada iteración.

Cada iteración del entrenamiento comienza con la ejecución del *forward propagation* y finaliza con el cómputo de la función de pérdida. A continuación, se lleva a cabo un proceso conocido como *backward*, que consiste en realizar operaciones que se inician a la salida de la FNN y retroceden hasta su entrada. La primera de ellas consiste en calcular el gradiente de la función de pérdida respecto a los parámetros  $\theta$  mediante el algoritmo de *backpropagation*. Posteriormente, el optimizador ajusta dichos parámetros para reducir la pérdida. Por lo tanto, el entrenamiento de una FNN puede formularse como un problema de optimización.

### 2.2.1. Función de pérdida

La función de pérdida es uno de los elementos fundamentales a definir para llevar a cabo el entrenamiento de una FNN. Su elección está principalmente condicionada al tipo de tarea que debe resolver el modelo. En este sentido, el presente desarrollo está orientado a la resolución de problemas de regresión lineal multi-variada. Para este tipo de tareas, se emplea la función de pérdida que calcula el error cuadrático medio, conocida como MSE por su sigla en inglés (*Mean Squared Error*) [2]. Este procedimiento se enmarca dentro de un contexto de aprendizaje supervisado, en el que el modelo aprende a partir de ejemplos etiquetados para minimizar la discrepancia entre las predicciones y los valores reales.

### 2.2.2. Algoritmo de *backpropagation*

En el contexto del aprendizaje automático, el algoritmo de *backpropagation* se emplea para calcular el gradiente de la función de pérdida con respecto a los parámetros del modelo [2]. Basándose en la combinación de las derivadas para una secuencia de operaciones mediante la regla de la cadena, calcula las derivadas de las salidas con respecto a las entradas de la red directamente en el grafo computacional. Este procedimiento permite determinar la sensibilidad de la función de pérdida frente a cada parámetro de la red. El cálculo se realiza de forma iterativa, propagando los errores desde la capa de salida hacia atrás hasta la capa de entrada.

### 2.2.3. Optimización

Una red neuronal es no lineal, lo que hace que la mayoría de las funciones de pérdida adquieran un carácter no convexa [2]. Como consecuencia, pueden aparecer puntos de silladura, mínimos locales o regiones con gradientes abruptos, lo que dificulta el proceso de optimización y la convergencia hacia una solución adecuada.

Para mitigar estos inconvenientes, se emplean optimizadores iterativos basados en el gradiente de la función de pérdida con respecto a los parámetros del modelo. Estos algoritmos permiten reducir su valor progresivamente hasta alcanzar

niveles muy bajos, lo que mejora el desempeño de la red durante el entrenamiento [2], aunque no garantizan la convergencia a un mínimo global.

En particular, en este trabajo se empleó el algoritmo de optimización L-BFGS (Broyden–Fletcher–Goldfarb–Shanno de memoria limitada), un método quasi-newtoniano de segundo orden diseñado para optimizar funciones con un gran número de parámetros, como los que surgen en el entrenamiento de modelos de aprendizaje profundo.

## 2.3. Modelado de PINN

Por definición, PINN es una red neuronal entrenada como función de aproximación para resolver sistemas EDP mediante el método de residuos. Es decir, las ecuaciones actúan como restricciones físicas incorporadas en la función de pérdida durante el entrenamiento de la red [11].

En todo modelo PINN, se toma como punto de partida la siguiente EDP 2.2 (el detalle del sistema EDP se desarrolla en el capítulo 3) [1]:

$$\begin{aligned}\mathcal{N}[y(\mathbf{x}, t; \beta)] &= q(\mathbf{x}, t) & \forall(\mathbf{x}, t) \in \Omega \times (0, T] \\ y(\mathbf{x}, t) &= h(\mathbf{x}, t) & \forall(\mathbf{x}, t) \in \partial\Omega \times (0, T] \\ y(\mathbf{x}) &= g(\mathbf{x}) & \forall(\mathbf{x}, t) \in \Omega \times \{t = 0\}\end{aligned}\tag{2.2}$$

donde  $\mathbf{x} \in \mathbb{D}$  representa los puntos en el dominio espacial para el instante de tiempo  $t$ . La función  $\mathcal{N}[\cdot]$  es el operador diferencial, dependiente de  $y$  y sus derivadas parciales sucesivas,  $q$  contiene los términos no homogéneos de la EDP,  $\Omega$  es el dominio espacial y  $\partial\Omega$  su frontera. Los parámetros de la EDP están dados por  $\beta = [\beta_1, \beta_2, \dots, \beta_m]$ . Las funciones  $h$  y  $g$  son las condiciones de borde (de tipo Dirichlet, Neumann o combinada [13]) y condiciones iniciales, respectivamente. En PINN, la solución exacta  $y$  es reemplazada por la salida de la red neuronal  $\hat{y}$ , de forma que la ecuación 2.2 resulta ser distinta de cero. Es decir, los residuos de dicha ecuación no son nulos.

### 2.3.1. Puntos de colocación

Los puntos de colocación [13] son ubicaciones específicas dentro del dominio espacio-temporal donde se computa la función de pérdida. Su selección y distribución influyen significativamente la precisión y eficiencia del entrenamiento, por lo que se definen distintas estrategias de muestreo que permiten cubrir adecuadamente el dominio, en función de los principios físicos que rigen el problema. Desde la perspectiva del aprendizaje profundo, estos puntos conforman los conjuntos de datos utilizados para entrenar la red neuronal.

El modelo de viga unidimensional considerado en este trabajo implica que el dominio espacial esté definido por una única coordenada  $x$ . Bajo esta premisa, se construyen los conjuntos de datos a partir de las variables independientes del modelo físico. Las coordenadas en el dominio espacio-temporal se organizan en 2.3:

$$\begin{aligned}\text{Dominio EDP} &= \{x_i, t_i\}^{N_d} & \forall(x, t) \in \Omega \times (0, t_f] \\ \text{Frontera EDP} &= \{x_i, t_i\}^{N_b} & \forall(x, t) \in \partial\Omega \times (0, t_f] \\ \text{Condiciones iniciales EDP} &= \{x_i\}^{N_{ci}} & \forall(x, t) \in \Omega \times \{t = 0\}\end{aligned}\tag{2.3}$$

donde  $N_{edp}$ ,  $N_b$  y  $N_{ci}$  denotan la cantidad de puntos en el dominio de las ecuaciones de gobierno, en su frontera y para sus condiciones iniciales, respectivamente. Para definir su distribución, se emplean distintas estrategias de muestreo. En el caso del dominio de las ecuaciones de gobierno se realiza un muestro del tipo hipercubo latino, conocido como LHS por su sigla en inglés (*Latin Hypercube Sampling*). Se trata de un método estadístico que genera una muestra casi aleatoria de valores a partir de distribuciones multivariadas. En cambio, para los puntos correspondientes a la frontera y a las condiciones iniciales, se recurre a muestreo aleatorio simple.

### 2.3.2. Función de pérdida

A continuación, se definen las funciones de pérdida a partir de los residuos de la ecuación 2.2 en 2.4:

$$\begin{aligned}
 \mathcal{L}_d &= \frac{1}{N_d} \sum_{i=1}^{N_d} |\mathcal{N}[\hat{y}(x_i, t_i)] - q(x_i, t_i)|^2 & \forall (\mathbf{x}, t) \in \Omega \times (0, T] \\
 \mathcal{L}_b &= \frac{1}{N_b} \sum_{i=1}^{N_b} |\hat{y}(x_i, t_i) - h(x_i, t_i)|^2 & \forall (\mathbf{x}, t) \in \partial\Omega \times (0, T] \\
 \mathcal{L}_{ci} &= \frac{1}{N_{ci}} \sum_{i=1}^{N_{ci}} |\hat{y}(x_i) - g(x_i)|^2 & \forall (\mathbf{x}, t) \in \Omega \times \{t = 0\} \\
 \mathcal{L}_{data} &= \frac{1}{N_{data}} \sum_{i=1}^{N_{data}} |\hat{y}(x_i, t_i) - y(x_i, t_i)|^2 & \forall (\mathbf{x}, t) \in \Omega \times (0, T]
 \end{aligned} \tag{2.4}$$

donde  $\mathcal{L}_d$  penaliza el residuo de la ecuación de gobierno. Las pérdidas  $\mathcal{L}_b$  y  $\mathcal{L}_{ci}$  actúan como restricciones para que las predicciones de la PINN satisfagan las condiciones de borde y las condiciones iniciales; mientras que  $\mathcal{L}_{data}$  corresponde a las pérdidas entre la respuesta de la red neuronal y datos rotulados. Su consideración es opcional para la resolución del problema directo, pero necesaria cuando se trabaja el problema inverso.

El siguiente paso consiste en computar la función de pérdida total en 2.5:

$$\mathcal{L} = \lambda_1 \mathcal{L}_d + \lambda_2 \mathcal{L}_b + \lambda_3 \mathcal{L}_{ci} + \lambda_4 \mathcal{L}_{data} \tag{2.5}$$

donde  $\lambda = [\lambda_1, \lambda_2, \lambda_3, \lambda_4]$  representa los factores de ponderación de las funciones de pérdida parciales. De esta manera, las ecuaciones 2.4 y 2.5 constituyen la base sobre la que se define y entrena un modelo PINN.

### 2.3.3. Esquema principal

La solución aproximada para una cañería sometida a vibraciones mecánicas, utilizando PINN, se expresa como  $\hat{y}(w(x, t), \psi(x, t); \theta)$ . En esta formulación,  $w(x, t)$  y  $\psi(x, t)$  representan las variables dependientes del modelo físico de una viga unidimensional del tipo Timoshenko [25], correspondientes a los desplazamientos y rotaciones transversales respecto de su eje principal.

El modelo de red neuronal utilizado en este trabajo corresponde a una red FNN tipo *fully connected*, compuesta por  $K$  capas. La capa de entrada recibe a los conjuntos de puntos de colocación de la ecuación 2.3, mientras que la capa de salida

$K$  representa la aproximación de la solución, es decir,  $\hat{y}$ . Una vez definida la arquitectura de la red neuronal, se completa el esquema de los modelos PINN a evaluar en este trabajo, tal como se muestra en la figura 2.1.

El primer paso consiste en calcular las funciones de pérdida asociadas al sistema EDP, de acuerdo con las ecuaciones 2.4 y 2.5. Para ello, es necesario computar las derivadas parciales de la solución aproximada. Gracias al algoritmo de *backpropagation*, estas derivadas pueden obtenerse directamente a partir del grafo computacional, lo que permite el uso de expresiones explícitas. Esto elimina los errores de truncamiento y facilita la implementación del modelo PINN. Finalmente, se emplea un optimizador basado en gradientes para ajustar los parámetros  $\theta$ .

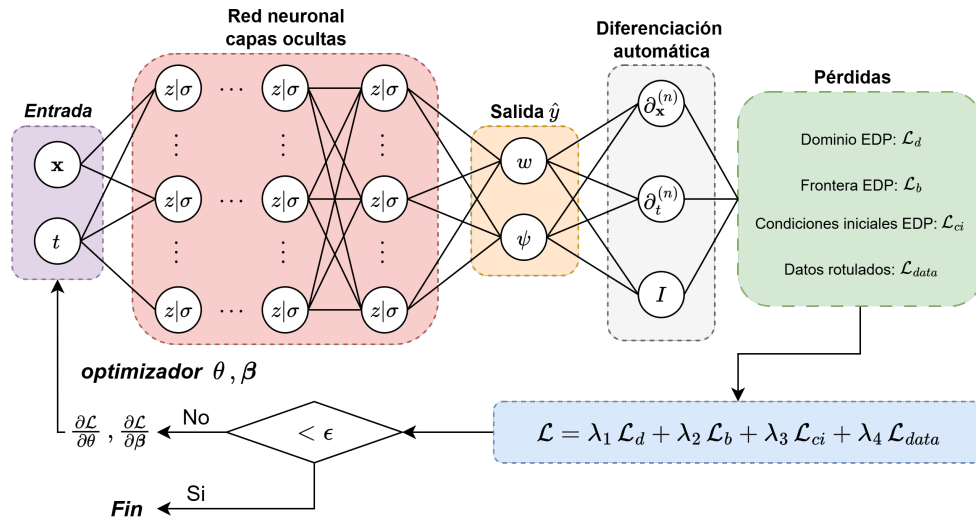


FIGURA 2.1. Esquema de una PINN para casos de vibraciones mecánicas.

En el proceso descrito, es importante considerar las diferencias entre la resolución del problema directo y del inverso. En el primer caso, no se requiere el uso de conjuntos de datos externos, por lo que no es necesario computar la función de pérdida asociada a los datos,  $\mathcal{L}_{data}$ . En consecuencia, solo se ajustan los parámetros de la red neuronal  $\theta$ .

En cambio, al abordar el problema inverso, el objetivo es estimar coeficientes desconocidos del sistema EDP, representados por  $\beta$ . En este trabajo, dichos coeficientes corresponden a las elasticidades de los soportes de la viga unidimensional. Para realizar el entrenamiento de la red neuronal, además del empleo de los conjuntos de puntos de colocación, se utilizan conjuntos de datos rotulados generados mediante simulación numérica en puntos específicos del dominio espacial. Esto hace necesario calcular la función de pérdida entre las predicciones de la red neuronal y los datos rotulados, a fin de ajustar tanto  $\theta$  como  $\beta$ .



## Capítulo 3

# Diseño e implementación

Este capítulo describe el diseño de los modelos PINN utilizados para evaluar cañerías sometidas a vibraciones mecánicas. Se incluye el desarrollo de los modelos físicos correspondientes a configuraciones de vigas unidimensionales, seleccionadas según el tipo de prueba realizada. Asimismo, se establecen los criterios para la construcción de los conjuntos de datos. En cuanto a la ejecución de los ensayos, se presenta la estructura del código diseñada para permitir la realización de múltiples pruebas, junto con la metodología empleada para el registro y análisis de los resultados experimentales.

### 3.1. Desarrollo de los modelos físicos

Los sistemas mecánicos suelen estar compuestos por elementos estructurales con masa y elasticidad distribuidas, lo que permite considerarlos como sistemas continuos con infinitos grados de libertad. Las vibraciones en este tipo de estructuras están gobernadas por EDPs, cuyas variables dependen tanto del tiempo como de las coordenadas espaciales.

Una cañería puede modelarse como un elemento estructural cuya longitud es considerablemente mayor que las dimensiones de su sección transversal. Cuando está sometida a fuerzas periódicas, se generan vibraciones mecánicas que inducen tensiones internas. Para determinar la distribución de estas tensiones, es necesario considerar las deformaciones provocadas por las vibraciones, que se calculan aplicando los principios fundamentales de la resistencia de materiales.

Se consideran dos tipos de vigas, diferenciadas según el enfoque utilizado para calcular las deformaciones provocadas por fuerzas externas. La primera es la viga tipo Euler-Bernoulli [25], empleada para evaluar las respuestas básicas esperables mediante el modelo PINN. La segunda corresponde a la viga tipo Timoshenko, seleccionada para abordar los objetivos específicos de este trabajo. A continuación, se describen sus características principales.

La viga tipo Euler-Bernoulli respeta el comportamiento descrito por la teoría homónima, en el que la región central de la viga no presenta fuerza cortante y está sometida a un momento flector constante. Esta situación se denomina flexión pura [27]. Las deformaciones inducidas por las vibraciones se calculan ignorando los efectos de la inercia rotacional y del esfuerzo cortante transversal. De esta forma, las secciones transversales de la viga permanecen normales al eje neutro durante la deformación. La distribución de esfuerzos internos debida a la flexión pura se determina a partir de la deformación de la viga, representada por los desplazamientos  $w(x, t)$ .

En este contexto, el modelo de viga propuesto por Timoshenko [25] se caracteriza por incorporar los efectos de la inercia rotacional y del esfuerzo cortante transversal. Como consecuencia, las secciones transversales de la viga no permanecen perpendiculares al eje neutro durante la deformación, lo que exige el cálculo conjunto del desplazamiento transversal  $w(x, t)$  y la rotación de la sección transversal  $\psi(x, t)$ . En este trabajo se desarrollaron dos modelos físicos basados en esta formulación, a saber:

- Un modelo físico específico, que se empleó para desarrollar la plataforma de trabajo y evaluar las respuestas dinámicas de los modelos PINN.
- Un modelo físico general, destinado emular la configuración y las condiciones a las que puede estar sometido un tramo de cañería.

Las ecuaciones utilizadas para describir los fenómenos de vibraciones mecánicas se expresaron en función de sus respectivos residuos. Esta formulación permite calcular las pérdidas parciales, tal como se indica en la ecuación 2.4. Además, es importante señalar que la resolución de los problemas se llevó a cabo en forma adimensional, con el propósito de evitar errores asociados a las escalas y mejorar la estabilidad numérica durante el proceso de cálculo. El desarrollo completo para la obtención de los términos adimensionales se presenta en el Apéndice A.

Con la intención de simplificar la notación matemática utilizada en este trabajo, se adoptó un criterio claro para representar las ecuaciones diferenciales. La diferenciación respecto de la variable espacial se expresa mediante la notación de Lagrange [28], en la cual cada derivada se indica con el símbolo prima (') aplicado a la variable dependiente. En cambio, la diferenciación respecto del tiempo se representa mediante la notación de Newton [28], que consiste en colocar un punto sobre la variable dependiente por cada derivada temporal. Asimismo se considera que  $W = W(X, T)$ ,  $\Psi = \Psi(X, T)$  y  $F_a = F_a(X, T)$ , donde:

- $X$  es la ubicación de un punto en la viga para el instante  $T$ , ambas variables independientes adimensionalizadas.
- Las funciones  $W$  y  $\Psi$  corresponden las expresiones adimensionales del desplazamiento transversal y rotacional, respectivamente.
- La función  $F_a$  representa la expresión adimensional de la fuerza de excitación.

### 3.1.1. Modelo físico específico

La elaboración del modelo físico específico parte del supuesto de viga esbelta, prismática, elástica y homogénea. Para la preparación de las respuestas básicas del modelo PINN, se consideró el caso de una viga sometida a una carga distribuida a lo largo de toda su longitud, tal como se muestra en la figura 3.1.



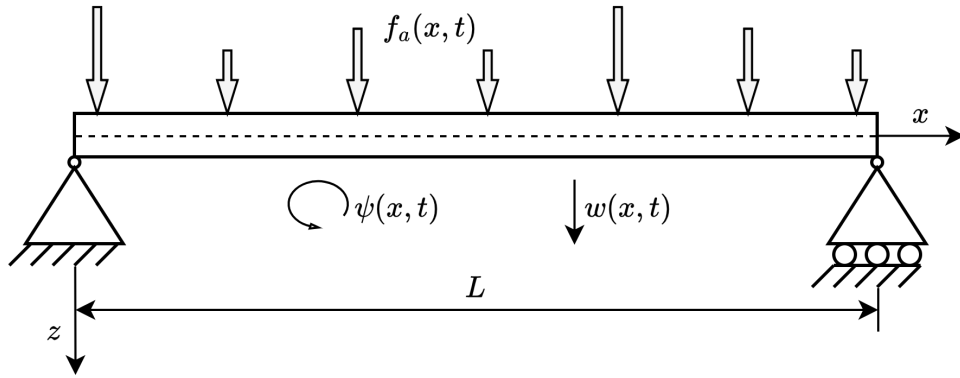


FIGURA 3.1. Esquema del modelo físico específico.

A continuación se formulan las ecuaciones que describen el comportamiento dinámico de la viga descrita en la figura 3.1 de forma adimensional (ver Apéndice A) [23]. En primer lugar se muestran las EDPs en 3.1:

$$r_d^{w1} = \Psi' - W'' + \ddot{W} - F_a^1 \quad (3.1a)$$

$$r_d^{\psi1} = -W' - \Psi'' + \Psi + \ddot{\Psi} \quad (3.1b)$$

Las ecuaciones descritas se complementan con los siguientes residuos de las condiciones de borde en 3.2 y 3.3, que responden al caso de una viga con los desplazamientos y rotaciones restringidos en ambos extremos [23]:

- En  $X = 0$ ,

$$r_{b_L}^{w1} = W(0, T) \quad r_{b_L}^{\psi1} = \Psi(0, T) \quad (3.2)$$

- En  $X = X_f$ ,

$$r_{b_R}^{w1} = W(X_f, T) \quad r_{b_R}^{\psi1} = \Psi(X_f, T) \quad (3.3)$$

En las ecuaciones 3.1, 3.2 y 3.3, el valor de  $X_f$  representa la cota superior del intervalo del dominio espacial. El dominio de aplicación se describe en la subsección 3.2.2 junto con el detalle de las cotas de los intervalos de valores de  $X$  y  $T$ .

El residuo con las condiciones iniciales está dada por la ecuación 3.4 [23]:

$$r_{c1}^{w1} = W(X, 0) - \text{sen}(X) \quad r_{c1}^{\psi1} = \Psi(X, T) - \frac{\pi}{2} \cos(X) - X + \frac{\pi}{2} \quad (3.4a)$$

$$r_{c2}^{w1} = \dot{W}(X, 0) \quad r_{c2}^{\psi1} = \dot{\Psi}(X, 0) \quad (3.4b)$$

La fuerza de excitación considerada en este caso de estudio está dada por la expresión 3.5 [23]:

$$F_a^1 = \cos(T) - \frac{\pi}{2} \text{sen}(X) \cos(T) \quad (3.5)$$

En el caso de la resolución del problema inverso para el modelo físico específico, se adaptan las ecuaciones de gobierno para realizar dos tipos de pruebas. La primera consiste en incorporar el parámetro desconocido  $\alpha$  en la ecuación 3.1b, tal como en 3.6:

$$r_d^{\psi1i} = -W' - \Psi'' + \Psi + \alpha \ddot{\Psi} \quad (3.6)$$

donde  $\alpha$  forma parte del conjunto de parámetros entrenables de la red neuronal del modelo PINN. Para evaluar la eficacia en la resolución del problema inverso, se toma como referencia el caso en que  $\alpha = 1$ .

La segunda adaptación se realiza para aproximar la expresión de la fuerza de excitación. En este caso, se busca estimar los parámetros desconocidos  $\gamma$  y  $\theta$ , presentes en la ecuación 3.5, como se observa en 3.7

$$F_a^0 = \beta \quad (3.7a)$$

$$F_a^1 = \gamma - \theta \sin(X) \cos(T) \quad (3.7b)$$

donde  $\beta$ ,  $\gamma$  y  $\theta$  también forman parte del conjunto de parámetros entrenables de la red neuronal del modelo PINN. Se debe tener en cuenta que  $\beta \approx 0$ , ya que  $F_a^0 = 0$ . La evaluación del desempeño se realiza comparando la ecuación 3.5 con la obtenida mediante el proceso inverso, en cada punto  $(X, T)$  del dominio. La estimación de las fuerzas de excitación resulta ser un caso interesante de tratar. En tareas de monitoreo de vibraciones mecánicas, en ocasiones se intenta hallar la fuente que provoca las excitaciones en las cañerías.

Finalmente, se presentan las soluciones analíticas a la ecuación 3.1 en las ecuaciones 3.8, según lo indicado en [23]:

$$W(X, T) = \frac{\pi}{2} \sin(X) \cos(T) \quad (3.8a)$$

$$\Psi(X, T) = \cos(T) \left[ \frac{\pi}{2} \cos(X) + \left( X - \frac{\pi}{2} \right) \right] \quad (3.8b)$$

Estas soluciones se emplearon como referencia para evaluar el desempeño del modelo PINN en las pruebas del modelo físico específico.

### 3.1.2. Modelo físico general

La figura 3.2 muestra la configuración de la viga correspondiente al modelo físico general. Se conserva el supuesto de viga esbelta, prismática, elástica y homogénea. Sin embargo, los extremos presentan rigideces lineales ( $k_L$  y  $k_R$ ) y torsionales ( $k_{tL}$  y  $k_{tR}$ ). En adelante, para facilitar la nomenclatura, cualquier referencia que involucre por igual a cualquiera de ellas se la mencionará como  $k_\alpha$ .

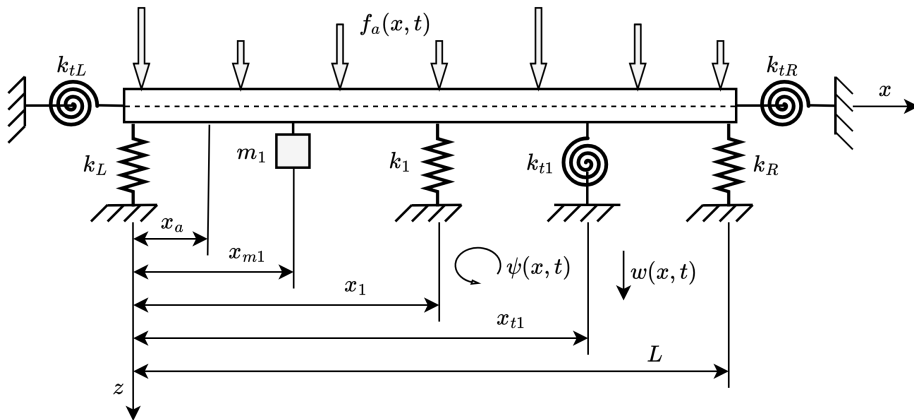


FIGURA 3.2. Esquema del modelo físico general.

A lo largo de la longitud de la viga se ubican una masa puntual  $m_1$ , un soporte de rigidez lineal  $k_1$  y un soporte de rigidez torsional  $k_{t1}$ . Las vibraciones son inducidas por la fuerza de excitación  $f_a(x, t)$ , que genera desplazamientos transversales  $w(x, t)$  y rotaciones de la sección transversal  $\psi(x, t)$ .

A continuación se formulan las ecuaciones que describen el comportamiento dinámico de la viga descrita en la figura 3.2 de forma adimensional (ver Apéndice A) [25]. En primer lugar se muestran las EDPs 3.9:

$$r_d^{w2} = -\Psi'' + W'' - \gamma_{bs} R_0^2 K_1 \delta(X - X_1) W - \gamma_{bs} R_0^2 [1 + M_1 \delta(X - X_{m1})] \ddot{W} + \gamma_{bs} R_0^2 F_a^2 \quad (3.9a)$$

$$r_d^{\psi2} = W' - [1 + K_{t1} \delta(X - X_{t1})] \Psi - \gamma_{bs} (R_0^2)^2 \ddot{\Psi} \quad (3.9b)$$

en las que los parámetros adimensionales  $\gamma_{bs}$  y  $R_0$  derivan de las propiedades del material y del radio de giro de la viga, respectivamente. Las coordenadas  $X_{m1}$ ,  $X_1$  y  $X_{t1}$  indican la posición de los elementos  $M_1$ ,  $K_1$  y  $K_{t1}$ , respectivamente. El efecto de cada elemento puntual en la deformación se modela mediante su producto con una función delta de Dirac.

Los residuos de las condiciones de borde asociadas a las ecuaciones 3.9 son específicos para cada extremo según las ecuaciones 3.10 y 3.11:

- En  $X = 0$ :

$$r_{bL}^{w2} = K_L W - \frac{1}{\gamma_{bs} R_0^2} (W' - \Psi) \quad 0 \leq K_L \leq \infty \quad (3.10a)$$

$$r_{bL}^{\psi2} = K_{tL} \Psi - \Psi' \quad 0 \leq K_{tL} \leq \infty \quad (3.10b)$$

- En  $X = X_f$ :

$$r_{bR}^{w2} = K_R W + \frac{1}{\gamma_{bs} R_0^2} (W' + \Psi) \quad 0 \leq K_R \leq \infty \quad (3.11a)$$

$$r_{bR}^{\psi2} = K_{tR} \Psi + \Psi' \quad 0 \leq K_{tR} \leq \infty \quad (3.11b)$$

El dominio de aplicación se describe detalladamente en la subsección 3.2.2 junto con el detalle de las cotas de los intervalos de valores de  $X$  y  $T$ .

Los residuos de las condiciones iniciales se expresan en 3.12:

$$r_{c1}^{w2} = W(X, 0) \quad r_{c1}^{\psi2} = \Psi(X, 0) \quad (3.12a)$$

$$r_{c2}^{w2} = \dot{W}(X, 0) \quad r_{c2}^{\psi2} = \dot{\Psi}(X, 0) \quad (3.12b)$$

La fuerza de excitación actúa a lo largo de toda la viga con valor constante en la coordenada espacial adimensional  $X$ , y está definida por la ecuación 3.13:

$$F_a^2 = F_a \operatorname{sen} \left( n\pi \frac{T}{T_f} \right) \quad (3.13)$$

donde  $F_a$  es la fuerza externa distribuida a lo largo de la viga de forma adimensionalizada. La constante define  $n$  la cantidad de ciclos en período  $T_f$ . Ambos valores se asignan al configurar el ensayo correspondiente.

Los términos de las ecuaciones 3.9a y 3.9b pueden ajustarse según el tipo de ensayo a los modelos PINN que se desee realizar, a saber:

- La estimación de  $K_\alpha$ , mediante la resolución de un problema inverso, implica que su valor no se especifique previamente.
- Los elementos puntuales no necesariamente se incluyen en todos los ensayos. En tales casos, sus valores se consideran nulos.
- En ciertos ensayos, la fuerza de excitación puede aplicarse en una región específica de la viga. En esas pruebas, su acción se localiza en la coordenada  $X_a$  y se modela mediante una función delta de Dirac como  $F_a^2 \delta(X - X_a)$ .

## 3.2. Consideraciones generales de los modelos PINN

A continuación se presentan los aspectos generales que caracterizan a los modelos PINN evaluados. Se detallan los componentes fundamentales, junto con aspectos particulares relevantes para su implementación. Asimismo, se explican los criterios adoptados para las etapas de entrenamiento y validación.

### 3.2.1. Características del *pipeline*

La elaboración del flujo de procesamiento (concepto conocido en inglés como *pipeline*) de un modelo PINN orientado al cómputo directo de las variables dependientes en estudio, el desplazamiento transversal  $W$  y la rotación de la sección transversal  $\Psi$ , requiere, como mínimo, la consideración de los siguientes aspectos:

1. La construcción de los conjuntos de datos de entrenamiento a partir de los subconjuntos de puntos de colocación.
2. La configuración de la red FNN del tipo *fully connected*, que incluye: el número de variables de entrada y salida, la cantidad de capas (profundidad), el número de neuronas por capa (ancho), la funciones de activación, el criterio de normalización de las variables de entrada y la inicialización de los parámetros entrenables.
3. Las definiciones de los residuos y los valores de los factores de ponderación  $\lambda$ .
4. La selección del algoritmo de optimización y el ajuste de sus hiperparámetros.
5. El desarrollo del proceso de entrenamiento, que incluye el cálculo de las funciones de pérdida parciales y total, y el ajuste de hiperparámetros.
6. La elaboración de gráficos que representen la evolución de las funciones de pérdida parciales y total a lo largo del entrenamiento.

La resolución del problema inverso presenta particularidades adicionales respecto al esquema utilizado para el cálculo directo de las variables dependientes. En

este caso, la estimación del valor desconocido  $K_\alpha$  requiere considerar los siguientes aspectos:

- La construcción de un conjunto de datos de entrenamiento a partir de datos rotulados, que se incorpora a los subconjuntos de puntos de colocación. Estos datos son generados sintéticamente a partir de simulaciones previas que proporcionan valores de desplazamientos y rotaciones en puntos específicos de la viga.
- La estimación de  $K_\alpha$  se considera un parámetro entrenable dentro del modelo.
- La necesidad de ajustar los hiperparámetros del algoritmo de optimización.
- La elaboración de un gráfico que muestre la evolución del valor estimado de la elasticidad durante el entrenamiento.

Dado que  $K_\alpha$  es un parámetro entrenable, su valor se determina al finalizar el proceso de entrenamiento del modelo PINN. Es decir, no se obtiene mediante una inferencia posterior a partir del modelo ya entrenado.

Los puntos mencionados dependen de la implementación algorítmica y la selección de valores de hiperparámetros en la configuración de cada ensayo. Sin embargo, existen aspectos que merecen un tratamiento previo especial. Se trata de los conjuntos de datos, la configuración de la arquitectura de la red neuronal, el cálculo de los residuos y los factores de ponderación.

### 3.2.2. Conjuntos de datos

La selección de los puntos de colocación constituye un aspecto fundamental en PINN. En la sección 2.3.1 se describió la conformación de los conjuntos de datos correspondientes al caso de una viga adimensional, según lo expresado en la ecuación 2.3. Asimismo, se establecen las estrategias de muestreo aplicadas a cada conjunto. Para el dominio de las ecuaciones de gobierno, se realiza un muestreo tipo LHS. En cambio, los puntos asociados a las condiciones de frontera y a las condiciones iniciales se obtuvieron mediante muestreo aleatorio simple.

Los modelos físicos presentados en la sección 3.1 exhibieron ciertas particularidades que fue necesario considerar para generar los subconjuntos de puntos de colocación. Se definieron las cotas de los dominios espacial y temporal utilizando variables adimensionales. Además, se contemplaron las propiedades específicas de las condiciones de borde, lo que permitió construir conjuntos de puntos diferenciados para cada extremo. Conforme a estas premisas, se configuraron los subconjuntos en 3.14:

$$\begin{aligned}
 \text{Dominio EDP} &= \{X_i, T_i\}^{N_d} & \forall (X, T) \in (0, X_f) \times (0, T_f] \\
 \text{Frontera izquierda EDP} &= \{X_i, T_i\}^{N_{bL}} & \forall (X, T) \in (X = 0) \times (0, T_f] \\
 \text{Frontera derecha EDP} &= \{X_i, T_i\}^{N_{bR}} & \forall (X, T) \in (X = X_f) \times (0, T_f] \\
 \text{Condiciones iniciales EDP} &= \{X_i\}^{N_{ci}} & \forall (X, T) \in [0, X_f] \times T = 0
 \end{aligned} \tag{3.14}$$

donde  $X_f$  y  $T_f$  representaron los valores correspondientes a la cota superior de los intervalos de los dominios espacial y temporal, según el tipo de modelo físico considerado. Para el modelo físico específico, se estableció que  $X_f = \pi$  y  $T_f = 1$ . En cuanto al modelo físico general, se adoptó el valor de  $X_f = 1$ , mientras que

en el dominio temporal abarcó un rango de valores de entre  $T_f = 2876,97$  y  $T_f = 5753,94$ , según el ensayo realizado. Cada uno de estos conjuntos de puntos fueron empleados para computar el residuo correspondiente. De acuerdo con la ecuación 3.14, se definió una función de pérdida específica para las condiciones de borde en cada extremo de la viga.

En relación con la estrategia de *batch training* (configuración del conjunto de datos para la realización del entrenamiento por lotes), en todos los ensayos se empleó la modalidad *full batch*, que implica el procesamiento del conjunto completo de datos en cada iteración del entrenamiento. Esta elección respondió al fundamento algorítmico del optimizador L-BFGS, mencionado en la subsección 2.2.3 [12].

En cuanto al uso de datos rotulados, se generaron de forma sintética y su origen dependió del modelo físico considerado. Para el modelo físico específico, se contó con la solución analítica presentada en la ecuación 3.8. En cambio, para el modelo físico general se recurrió a soluciones obtenidas por el MEF. En ambos casos, se conformaron dos tipos de conjuntos de datos según su propósito.

El primero se empleó durante los entrenamientos destinados a resolver el problema inverso, simulando la disponibilidad de mediciones de desplazamientos y torsiones transversales en puntos específicos de la viga. Para aproximar la toma de datos de sensores reales, se añadió ruido blanco en torno a cada punto. El segundo se utilizó como conjunto valores *ground-truth* (valores considerados como correctos) para servir de base de comparación en la evaluación del desempeño de los modelos PINN.

### 3.2.3. Configuración de la red FNN

Tal como se indicó en la sección 2.3.3, el modelo de red neuronal empleado en este trabajo corresponde a una red FNN tipo *fully connected*. Las variables de entrada consideradas son la coordenada adimensional  $X$  sobre la viga y el instante temporal adimensional  $T$  correspondiente. En cuanto a las variables de salida, se obtienen el desplazamiento transversal  $W$  y la rotación transversal  $\Psi$ .

La configuración de la red neuronal debe ser tal que no permita que los gradientes adquieran valores extremadamente pequeños, fenómeno conocido en inglés como *vanishing gradient*. Ocurre durante el proceso de entrenamiento al aplicar el algoritmo de *backpropagation* a través de las distintas capas, lo que limita significativamente el aprendizaje en las capas iniciales. Esta disminución se origina en la multiplicación repetida de derivadas de valor muy reducido dentro de la regla de la cadena.

Como consecuencia, se producen variaciones mínimas en los pesos de las capas más cercanas a la entrada que dificultan la capacidad de aprendizaje del modelo. A continuación, se identifican las principales causas que contribuyen a este fenómeno, junto con las estrategias implementadas para mitigar sus efectos adversos en el proceso de optimización.

El primer aspecto a considerar durante los ensayos es el grado de profundidad de la red. A medida que se incrementa el número de capas, se produce una mayor cantidad de multiplicaciones en el cálculo del gradiente, lo que conlleva una reducción significativa en su magnitud. Esta situación afecta negativamente el proceso de aprendizaje, especialmente en las capas iniciales. Por esta razón, las estructuras de red desarrolladas privilegian el ancho sobre la profundidad, con el

objetivo de mitigar los efectos del *vanishing gradient* y favorecer una optimización más eficiente.

Otro factor que influye en la capacidad de representación funcional de PINN es la elección de la función de activación, ya que determina el grado de no linealidad y, en consecuencia, afecta la capacidad de ajuste de la red neuronal. Conviene evitar aquellas funciones cuya derivada de orden superior sea discontinua o nula, dado que las ecuaciones empleadas para el análisis de vibraciones mecánicas incluyen derivadas de orden superior. Si esta condición no se cumple, la aplicación de la regla de la cadena durante la actualización de los parámetros puede derivar en un proceso de optimización ineficaz. La función tangente hiperbólica representa la opción más utilizada en PINN, adoptándola como la función de activación en las pruebas realizadas.

Finalmente, resulta fundamental asignar valores iniciales a los parámetros de la red neuronal antes de comenzar el entrenamiento. Una inicialización adecuada contribuye a evitar problemas como el *vanishing gradient*, el *exploding gradient* (los gradientes adquieren valores extremadamente grandes) y la saturación de la función de pérdida. En este trabajo, las pruebas se ejecutaron con la inicialización del tipo `Xavier`, ya que ofrece un equilibrio en la propagación de los gradientes y favorece la estabilidad del proceso de optimización.

### 3.2.4. Aspectos específicos de los modelos PINN

Esta sección presenta tres aspectos que requirieron definiciones específicas para su implementación en los modelos PINN. El primero se relaciona con la organización de los residuos, necesaria para el cálculo posterior de las pérdidas parciales. El segundo corresponde al método adoptado para aproximar las funciones delta de Dirac incluidas en la ecuación 3.9. Finalmente, se expone el criterio utilizado en la aplicación de los factores de ponderación requeridos para el cálculo de la pérdida total.

En la subsección 2.2.1 se mencionó que las pérdidas parciales se obtienen mediante el cálculo del MSE, a través una implementación algorítmica predefinida. Recibe como entrada los residuos calculados en cada punto de colocación, definidos previamente en la sección 3.1. Para que el cálculo sea correcto, los residuos deben estar organizados en vectores que respetan la estructura establecida en la ecuación 3.14.

Los residuos de los modelos físicos se agrupan en 3.15:

$$\begin{aligned}
 \text{Dominio EDP} : \mathbf{r}_d^j &= [r_d^{w_j}, r_d^{\psi_j}] \\
 \text{Frontera izquierda EDP} : \mathbf{r}_{b_L}^j &= [r_{b_L}^{w_j}, r_{b_L}^{\psi_j}] \\
 \text{Frontera derecha EDP} : \mathbf{r}_{b_R}^j &= [r_{b_R}^{w_j}, r_{b_R}^{\psi_j}] \\
 \text{Condiciones iniciales EDP} : \mathbf{r}_c^j &= [r_{c_1}^{w_j}, r_{c_1}^{\psi_j}, r_{c_2}^{w_j}, r_{c_2}^{\psi_j}]
 \end{aligned} \tag{3.15}$$

donde el supraíndice  $j$  indica el tipo de modelo físico considerado. Cuando  $j = 1$ , se hace referencia a los residuos correspondientes al modelo físico específico. En cambio, si  $j = 2$  los residuos se asocian al modelo físico general.

Otra cuestión relevante fue el método utilizado para calcular las funciones delta de Dirac que aparecen en los residuos de la ecuación 3.9. Para ello, se empleó una



aproximación mediante una función gaussiana, adaptada al modelo físico general según 3.16:

$$G(X) = e^{-\frac{(X-X_j)^2}{2\sigma^2}} \quad (3.16)$$

donde  $X_j$  representó la posición del elemento puntual sobre la viga, coincidiendo con el centro de la campana gaussiana. La constante  $\sigma$  correspondió al desvío estándar y determinó el ancho de la campana. Se procuró que su valor fuera lo más pequeño posible, de modo que la influencia de la función se limitara a los puntos de la viga cercanos a  $X_j$ .

Por último, resta mencionar el valor de los factores de ponderación. En este trabajo, a los elementos de  $\lambda$  se les asignaron valores unitarios.

### 3.3. Características del código

El desarrollo del código para la realización de los ensayos contempla dos aspectos fundamentales. El primero corresponde al uso del lenguaje de programación Python y sus herramientas asociadas para la elaboración de los modelos PINN. El segundo se refiere a la estructura del código, diseñada para facilitar la ejecución de pruebas ante la variabilidad de los modelos físicos y de los hiperparámetros.

Python se caracteriza por una sintaxis intuitiva y concisa, lo que resulta especialmente útil en la construcción de algoritmos y modelos complejos que requieren claridad en el código. Además, dispone de una amplia variedad de bibliotecas y *frameworks*, así como de una comunidad activa de desarrolladores, investigadores y profesionales que contribuyen al mantenimiento y evolución de proyectos de código abierto.

Otro aspecto relevante es su capacidad de integración con otros lenguajes, lo que permite utilizar Python para escribir código de alto nivel orientado al análisis y la visualización de datos, mientras que lenguajes como C o C++ se emplean en tareas que demandan una mayor eficiencia computacional. Estas características han consolidado a Python como un lenguaje estándar en el ámbito de la inteligencia artificial y la ciencia de datos [29].

El otro aspecto relevante fue el diseño de la estructura del código. Este trabajo presentó variabilidad tanto en la configuración de los modelos físicos como en la de los modelos PINN, lo que planteó la necesidad de desarrollar un esquema modular con capacidad para realizar pruebas sin modificar el código. Esta organización permitió registrar no solo los resultados obtenidos, sino también todas las configuraciones utilizadas. Además, se implementó una metodología específica para llevar el registro de resultados y configuraciones de ambos tipos de modelos evaluados, mediante herramientas particulares de gestión.

A continuación se describen los elementos que hicieron posible la implementación, el registro y la evaluación de los modelos PINN correspondientes a cada ensayo realizado.

#### 3.3.1. Implementaciones del *framework* Pytorch

El *framework* PyTorch se utilizó como entorno principal para el desarrollo de los modelos PINN. Se trata de una biblioteca de código abierto escrita principalmente en Python y C++, orientada al desarrollo de modelos de *machine learning* y, en



particular, de *deep learning*. En la actualidad, PyTorch se ha consolidado como una herramienta de referencia en el campo de la inteligencia artificial, ya que proporciona todos los componentes necesarios para construir un entorno completo de desarrollo en esta área tecnológica. Entre sus principales características se destacan:

- Compatibilidad con otros *frameworks* del ecosistema Python. Su sintaxis, similar a la de NumPy [30] (biblioteca ampliamente utilizada en computación científica para operar con matrices multidimensionales), facilita la integración y el aprendizaje.
- Disponibilidad de herramientas para construir el *pipeline* de un modelo de *deep learning*, que incluye la carga de datos, el preprocesamiento, el entrenamiento y la evaluación.
- Integración con CUDA (*Compute Unified Device Architecture*) [31], la plataforma de computación paralela desarrollada por NVIDIA, que permite aprovechar el uso de GPU para acelerar los cálculos computacionales.
- Entornos especializados para distintos tipos de datos y tareas, optimizados para dominios específicos, lo que facilita el desarrollo de soluciones completas y eficientes.

Actualmente, PyTorch no dispone de un entorno especializado para trabajar directamente con modelos PINN. No obstante, se emplean diversas funcionalidades que permiten implementar los distintos componentes del *pipeline* y sus variantes, tal como se describe en la subsección 3.2.1.

El módulo `torch.nn` proporciona una colección de clases y funciones diseñadas para construir y entrenar redes neuronales de forma estructurada y eficiente. Permite seleccionar distintos tipos de capas junto con sus propiedades, así como definir funciones de activación y funciones de pérdida, cada una con sus respectivas configuraciones. Por su parte, la clase `nn.Module` se encarga de definir la arquitectura completa de la red, ya que agrupa las capas, establece el flujo de datos (*forward*) y gestiona los parámetros entrenables. Esto incluye los métodos necesarios para ejecutar la inicialización del tipo `Xavier`.

Respecto al cálculo de las pérdidas, se utilizó el método `torch.nn.MSELoss`, que aplica la métrica MSE. Este método fue diseñado para recibir como entrada un tensor con los valores predichos por el modelo y otro con los valores objetivo. Dado que se trabaja con residuos, el tensor de valores objetivo se definió como nulo. Otra consideración importante es la metodología de cálculo que utiliza. Como el método permite procesar tensores de dimensiones  $n \times m$ , la pérdida se calculó según 3.17:

$$\text{MSE} = \frac{1}{n \cdot m} \sum_{i=1}^n \sum_{j=1}^m (y_{ij} - \hat{y}_{ij})^2 \quad (3.17)$$

donde  $n$  representa la cantidad de puntos en cada subconjunto de colocación o en el conjunto de datos externos. El valor de  $m$  es igual a 2, ya que corresponde a las variables de salida del modelo físico, que son el desplazamiento transversal  $W$  y la rotación transversal  $\Psi$ .

La ejecución del algoritmo de *backpropagation* se lleva a cabo mediante el esquema de diferenciación automática de PyTorch, denominado `autograd`. Este componente tiene como función principal la construcción dinámica de un grafo computacional que representa las operaciones realizadas sobre tensores, lo que permite el cálculo automático de derivadas parciales. Esta capacidad resulta esencial para obtener los residuos requeridos en el cómputo de las pérdidas parciales, ya que dichas pérdidas dependen de derivadas de primer y segundo orden de las variables de salida respecto de las de entrada de la red neuronal, conforme a las EDPs descritas en la sección 3.1.

Además, `autograd` proporciona el método `backward`, utilizado para calcular los gradientes de la función de pérdida con respecto a los parámetros de la red neuronal durante el proceso de entrenamiento.

Finalmente, se menciona el proceso de optimización de los parámetros del modelo, llevado a cabo mediante el módulo `torch.optim` de PyTorch. Este componente proporciona una amplia variedad de algoritmos de optimización diseñados para ajustar los parámetros entrenables de modelos de *deep learning*. A partir de los gradientes de la función de pérdida, calculados mediante el método `backward`, los optimizadores actualizan los parámetros del modelo conforme a la estrategia de optimización seleccionada.

Entre los algoritmos disponibles, se empleó el optimizador `optim.LBFGS`, que implementa el método de Broydon-Fletcher-Reeves-Shanno (BFGS) de memoria limitada `L-BFGS` cuya tasa de convergencia es superlineal. Debido a la naturaleza de su algoritmo, este optimizador necesita reevaluar la función de pérdida varias veces por iteración. Por ello, es necesario implementar una función auxiliar denominada `closure`, que encapsula tanto el cálculo de la pérdida como la ejecución del algoritmo de *backpropagation*.

Una limitación relevante de `LBFGS` es la imposibilidad de asignar tasas de aprendizaje diferenciadas a distintos grupos de parámetros. Esta característica puede representar una restricción en el contexto de problemas inversos abordados mediante PINNs, donde ciertos parámetros asociados al modelo físico (particularmente aquellos desconocidos) podrían requerir ajustes diferenciados durante el proceso de entrenamiento.

### 3.3.2. Estructura

La ejecución completa del *pipeline* de modelado PINN implicó la configuración de múltiples variables, ya que, además de los parámetros habituales en modelos de *deep learning*, se incorporaron aquellos propios de los modelos físicos. Este trabajo no tuvo como objetivo lograr un control exhaustivo de todos los posibles hiperparámetros, sino explorar distintas condiciones en los modelos físicos, lo que exigía modificar configuraciones entre ensayos. Para facilitar este proceso, se diseñó una estructura modular basada en clases y métodos de uso estándar, de modo que los ajustes necesarios al cambiar el tipo de prueba pudieran realizarse externamente o con una intervención mínima sobre los scripts de ejecución.

Se desarrolló un repositorio estructurado en dos bloques principales. El primero está dedicado al código fuente y su ejecución, mientras que el segundo agrupa toda la información utilizada o generada durante los ensayos. En ambos casos,

la organización respeta las tres instancias fundamentales que conforman el *pipeline* completo de modelado PINN, que incluye la generación de los conjuntos de datos, el entrenamiento del modelo y la evaluación de su desempeño.

El código se estructuró en tres partes, cada una con una función específica:

- `parameters`, en el que se incluyen archivos en formato JSON que permiten definir tanto las configuraciones del modelo físico como los hiperparámetros de los modelos PINN.
- `scripts`, que contiene los scripts preparados para ejecutar cada una de las etapas del entrenamiento de los modelos PINN.
- `src`, compuesto por módulos que agrupan clases y métodos necesarios para llevar a cabo las distintas fases del entrenamiento.

Dentro del directorio `parameters` se incluyen tres archivos en formato JSON, cada uno destinado a:

- La preparación de los conjuntos de datos utilizados en las etapas de entrenamiento y evaluación de desempeño (`param_data_predict.json`).
- La configuración estructural y funcional de los modelos PINN (`param_model.json`).
- La definición de los hiperparámetros empleados durante el proceso de entrenamiento (`param_train.json`).

Los `scripts` disponibles también se organizan en tres archivos, en concordancia con los archivos de configuración JSON. En concreto, el primero genera los conjuntos de datos (ya sea para entrenamiento o evaluación), el segundo ejecuta el proceso de entrenamiento del modelo PINN y el tercero realiza la evaluación del modelo previamente entrenado.

En el directorio `src` se incluyen los siguientes módulos:

- `module_data.py`, encargado de preparar los subconjuntos de puntos de colocación y datos externos, en el formato requerido para el cálculo de cada residuo y pérdida parcial.
- `module_pinn.py`, que proporciona las clases y métodos necesarios para construir el modelo PINN y entrenarlo en el contexto de la resolución del problema directo.
- `module_pinn_inv.py`, que ofrece las clases y métodos para construir el modelo PINN y entrenarlo en el marco de la resolución del problema inverso.
- `module_plot.py`, que incluye clases destinadas a la generación de gráficos durante la ejecución del *pipeline* completo de modelado PINN.

En lo que respecta a la información que interviene a lo largo de todo el *pipeline* de modelado PINN, su almacenamiento se organiza de la siguiente manera:

- `data`, donde se alojan los conjuntos de datos utilizados para el entrenamiento del modelo PINN, tanto en la resolución del problema directo como del inverso.

- *figures*, que contiene los gráficos generados durante el desarrollo completo del *pipeline* de modelado PINN.
- *metrics*, donde se registran los valores que adopta la función de pérdida total de cada modelo PINN a lo largo del entrenamiento.
- *models*, que almacena los valores finales de los parámetros de cada modelo PINN una vez concluido su entrenamiento.

### 3.4. Gestión de ensayos

Una particularidad de este trabajo fue la necesidad de registrar la respuesta física de cada modelo PINN ensayado en relación con la configuración correspondiente del modelo físico. Por un lado, se analizaron los desempeños desde la perspectiva de *machine learning*, evaluando métricas propias del entrenamiento. Sin embargo, también resultó fundamental que los resultados fueran coherentes con los principios físicos que rigen el problema abordado. La gestión del desarrollo de los ensayos se llevó a cabo mediante una integración complementaria entre la plataforma Notion [32] y el sistema de control de versiones Git, para aprovechar las fortalezas de cada una para registrar, organizar y versionar la información generada.

Notion es una plataforma de productividad todo-en-uno que permite crear, organizar y compartir contenido como notas, tareas, bases de datos, wikis, entre otros. El elemento central que facilitó la gestión de los ensayos fueron sus bases de datos no relacionales, que funcionan como tablas flexibles o colecciones de objetos, similares a estructuras tipo JSON. En cada una de ellas, cada fila corresponde a una página con propiedades (campos) que pueden contener texto, imágenes incrustadas, fechas, etiquetas, relaciones con otras filas de distintas tablas, entre otros elementos. Esta característica permitió registrar cada ensayo incluyendo los resultados de la respuesta física durante la evaluación de desempeño, las configuraciones del modelo y su vinculación con la documentación de respaldo.

Respecto a Git, su uso no se limitó a la gestión del desarrollo del código. Se utilizó para dejar registro en el repositorio del estado del código y la información generada al finalizar cada ensayos, con la misma referencia que la empleada en Notion. De esta forma, se resguardó la trazabilidad de configuración de modelos y registro de resultados.

El repositorio público propiedad del autor de este trabajo se encuentra alojado en la plataforma GitHub [33].

## Capítulo 4

# Ensayos y resultados

Este capítulo está dedicado a los ensayos realizados para las distintas configuraciones de modelos PINN. Se presentan las condiciones en las que se ejecutaron los ensayos, junto con la descripción de los recursos disponibles para su implementación y las limitaciones impuestas por cada formulación física al momento de reproducir fielmente el fenómeno estudiado. Se detallan las características de los casos seleccionados para evaluar los resultados obtenidos durante el proceso de entrenamiento. Finalmente, se muestran los resultados en función del modelo físico y se identifican los factores que afectaron la precisión y la coherencia en la representación de los fenómenos físicos.

### 4.1. Presentación general

El proceso de entrenamiento de los modelos PINN estuvo marcado por tres elementos que influyeron directamente en el alcance de los ensayos. El primero fue la capacidad de cómputo disponible, que actuó como factor limitante al momento de definir la complejidad posible del modelo. El segundo elemento corresponde a la amplia variedad de configuraciones que pueden adoptar los hiperparámetros, necesarias para que la red neuronal logre inferencias con coherencia física y alcance los objetivos de convergencia. Por último, se destacan las particularidades del fenómeno físico a representar, especialmente en el caso del modelo general, que exigieron ajustes adicionales en los modelos más allá de la configuración de hiperparámetros.

Al delinear las primeras configuraciones de modelos PINN, la consistencia en la representación de los fenómenos físicos durante las inferencias resultó ser un factor determinante en sus desarrollos. Esto motivó un estudio profundo de la física de vigas sometidas a vibraciones mecánicas, ya que fue necesario comprender las limitaciones que podría presentar una red FNN tipo *fully connected* al abordar este tipo de problemas, así como también identificar las adaptaciones requeridas en las ecuaciones de gobierno para lograr procesos de entrenamiento óptimos.

Por esta razón, se decidió evaluar inicialmente un modelo físico cuya formulación matemática había sido optimizada para resolver un problema específico de ingeniería, lo que permitió realizar una primera validación del enfoque basado en PINN. A partir de allí, se planteó un avance progresivo hacia la evaluación de un modelo físico de mayor generalidad.

En los siguientes apartados se describen los recursos computacionales utilizados y las consideraciones relacionadas a la configuración de los ensayos. También se

exponen las principales diferencias entre los modelos físicos específico y general, que influyeron tanto en la configuración de los modelos PINN como en la planificación de los ensayos.

#### 4.1.1. Recursos técnicos

La definición de los recursos computacionales disponibles establece un límite claro a la complejidad que puede alcanzar cualquier desarrollo en el campo de la IA. En este contexto, los modelos implementados y las pruebas realizadas se ajustaron a las capacidades técnicas disponibles para cumplir con los objetivos planteados en este trabajo.

Se utilizaron tanto plataformas desplegadas en la nube como recursos locales. En el primer caso, se recurrió al servicio básico de Google Colaboratory [34], un entorno gratuito basado en Jupyter Notebook, que en este trabajo se empleó exclusivamente dentro del alcance de acceso libre a capacidades de CPU y GPU. Su uso principal fue el siguiente:

- Compartir con directores y colaboradores externos la exploración de datos, esquemas de código y cálculos.
- Intercambiar cálculos y valores de parámetros físicos para simulaciones externas.

En relación con el uso de recursos locales, y considerando el tipo de pruebas realizadas, no se presentaron inconvenientes significativos derivados de limitaciones en la capacidad de cómputo. Una de las principales ventajas fue evitar las restricciones impuestas por las plataformas en la nube que, a pesar de ser gratuitas, limitan el tiempo de ejecución del código. Esto permitió mayor libertad para realizar pruebas sobre la arquitectura de la red y la configuración del algoritmo de optimización.

En cuanto al hardware utilizado, se contó con el siguiente equipamiento:

- GPU: Placa Nvidia RTX 4060, 16GB, con CUDA 12.0.
- CPU: Procesador Intel Core i7-1450HX.
- Memoria RAM: 32GB.

A continuación se describe el entorno de desarrollo utilizado durante la implementación de los modelos y la ejecución de los ensayos:

- Sistema operativo: Linux Debian [35].
- Editor de código: Visual Studio Code [36].
- Gestión de entornos de desarrollo: Conda [37].
- Control de versiones y colaboración: plataforma GitHub, utilizada para compartir el proceso de desarrollo y los ensayos con colaboradores y directores.

#### 4.1.2. Configuración de los ensayos

Cada ensayo representó un caso particular correspondiente a uno de los modelos físicos estudiados en este trabajo. Su preparación consistió, principalmente, en el ajuste de los hiperparámetros del modelo PINN correspondiente. Para ello, se llevaron a cabo pruebas iterativas orientadas a identificar la configuración que

mejor se adaptara al caso de estudio, que exigió equilibrar el entrenamiento óptimo de la red neuronal con la coherencia en la representación del fenómeno físico que se buscaba modelar.

En este proceso, el foco estuvo puesto en la definición de los subconjuntos de puntos de colocación y en el cómputo de los residuos, ya que estas tareas resultaron fundamentales para garantizar tanto la fidelidad física del modelo PINN como su capacidad de aprendizaje.

La configuración completa de los conjuntos de datos utilizados en las etapas de entrenamiento y evaluación del desempeño añadió complejidad a la definición de cada modelo PINN a ensayar. En cada prueba primero se revisaban y, si a merita, se ajustaban las cantidades de los subconjuntos de puntos de colocación, tal como se indica en la ecuación 3.14. No obstante, se presentaron situaciones en las que se contempló la incorporación de puntos en regiones acotadas del dominio. Para este trabajo, se trató de áreas longitudinales dadas por intervalos específicos en la coordenada  $X$  que cubrían todo el dominio temporal.

Esta decisión respondió a la necesidad de explorar alternativas que permitieran mejorar los resultados obtenidos o reforzar el área de influencia de ciertos elementos del modelo físico. Esta estrategia fue relevante en los ensayos correspondientes al modelo físico general. El tamaño del intervalo en la coordenada  $X$  estuvo determinado por el valor de la constante  $\sigma$  de la función gaussiana presentada en la ecuación 3.16, que era ajustado de forma iterativa en función de los resultados obtenidos en las sucesivas pruebas dentro del marco del caso en estudio.

Asimismo, se podían agregar puntos de colocación en aquellas regiones del dominio que se encontraban bajo la acción de elementos puntuales. Por sus características, este tipo de configuración se aplicó en las pruebas del modelo físico general. Al igual que en el procedimiento anterior, para cada ensayo se definió el valor la constante  $\sigma$ , que también se ajustaba de manera iterativa para hallar resultados óptimos.

Finalmente, se utilizaron datos rotulados para simular información proveniente de sensores ubicados en coordenadas específicas de la viga, dedicados a informar valores de desplazamientos y rotaciones. A diferencia de los casos anteriores, estos datos fueron modificados mediante la adición de ruido blanco. Su utilización se centró en las pruebas del problema inverso.

En lo que respecta a la arquitectura y a los residuos del modelo PINN, el ajuste de hiperparámetros en cada ensayo se organizó en dos secciones bien diferenciadas. Por un lado, se definieron las características de los parámetros del modelo físico a representar, que aplican directamente en los residuos. Este aspecto recibió especial atención en los ensayos relacionados con el modelo físico general, tanto para asignar valores a las rigideces como para establecer las características de la fuerza externa aplicada a la viga y a los elementos puntuales. Por otro lado, se determinaron los valores de los parámetros asociados a la arquitectura de la red neuronal.

Para completar el proceso de configuración, en cada ensayo se determinaron el valor de *learning rate* del optimizador `optim.LBFGS` y la cantidad de iteraciones de entrenamiento, también conocido por su denominación en inglés como *epochs*.

La implementación de archivos JSON específicos para modelos PINN (ver la subsección 3.3.2) no solo permitió configurar cada ensayo de manera estructurada,

sino que también facilitó el registro de los resultados obtenidos. Cada ejecución quedó vinculada a un commit específico en Git, asociado a su vez a una entrada en la base de datos de Notion, lo que permitió incorporar información complementaria, como gráficos y observaciones.

#### 4.1.3. Planificación según los modelos físicos

La planificación de los ensayos estuvo condicionada por las características particulares de cada tipo de modelo físico estudiado. Cada uno fue concebido para abordar el problema de las vibraciones mecánicas en vigas desde perspectivas distintas, lo que derivó en diferentes grados de complejidad entre sí. Esta distinción permitió establecer una estrategia de avance gradual en el estudio y desarrollo de los modelos PINN aplicado al tratamiento de vibraciones mecánicas. Además, facilitó una exploración más profunda de los alcances, fortalezas y limitaciones de este enfoque, ya que el nivel de dificultad de cada ensayo se correspondió con las exigencias propias del tipo de modelo físico considerado.

Por otra parte, se aprovechó el modelo físico específico para desarrollar los algoritmos e implementar los modelos PINN, de manera que fuera posible ejecutar múltiples pruebas mediante la configuración externa de sus parámetros. Esto permitió que las modificaciones necesarias en el código no dependieran de cambios operativos. Asimismo, se ajustaron los procedimientos para generar los gráficos correspondientes a la distribución de los subconjuntos de puntos de colocación en el dominio, así como las pérdidas parciales y las inferencias.

El modelo físico específico correspondió a un caso de estudio con un marco de condiciones bien definido. En particular, la viga presentó una configuración determinada y estuvo sometida a esfuerzos externos conocidos (ver la subsección 3.1.1). Este tipo de esquemas permite ajustar la representación matemática del fenómeno físico a las características de la técnica computacional empleada para su tratamiento, como fue el caso de la estrategia utilizada para adimensionar todos los parámetros involucrados en las ecuaciones correspondientes. Esta formulación ofreció ventajas para el desarrollo de la plataforma de trabajo y para iniciar la exploración de PINN en la temática abordada.

No obstante, también presentó ciertas limitaciones. Por un lado, no se contó con margen para experimentar con variaciones en la configuración del modelo físico. Por otro, solo se analizó una respuesta dinámica específica, sin posibilidad de evaluar el comportamiento ante diferentes frecuencias de excitación.

En cambio, el modelo físico general fue concebido para permitir la evaluación de diferentes condiciones de borde, características de las fuerzas externas y realizar la activación de elementos puntuales. Con esta aproximación, se buscó emular situaciones en las que cada tramo de una red de cañerías experimenta de manera diferenciada el fenómeno de las vibraciones mecánicas, ya sea por la presencia de soportes con distintos niveles de rigidez, la exposición a fuentes de excitación específicas o la incorporación de accesorios a lo largo de su longitud.

La implementación descrita en la subsección 4.1.2, facilitó la realización ágil de múltiples configuraciones mediante la modificación de valores clave que permitieron caracterizar el modelo físico a ensayar. Sin embargo, este esquema de



trabajo presentó desafíos, derivados de la complejidad de la representación matemática requerida y su interacción con las particularidades del modelo de red neuronal.

Por último, se detalla la metodología utilizada para evaluar el desempeño de cada modelo PINN. Para cada ensayo realizado, se planificó la ejecución de dos tipos de métricas del error. El primer tipo la define la ecuación 4.1 con el cálculo del error relativo  $L^2$ :

$$e_{L^2} = \frac{\|y - \hat{y}\|_2}{\|y\|_2} \quad (4.1)$$

El objetivo fue comparar cuantitativamente las distribuciones de desplazamientos y rotaciones *ground-truth* y aproximada. En una segunda instancia, se calculó a través de la ecuación 4.2 el error absoluto en cada punto  $i$  del dominio

$$e_{abs_i} = |y_i - \hat{y}_i| \quad (4.2)$$

donde  $i$  representa un punto  $(X, T)$  dentro del dominio considerado en el ensayo correspondiente. Este resultado permitió construir un mapa del error absoluto en todo el dominio, útil para identificar las zonas en las que el modelo PINN no logra representar con precisión el fenómeno físico. A partir de este análisis, fue posible definir estrategias de ajuste del modelo, como la incorporación de puntos de colocación adicionales.

## 4.2. Ensayos del modelo específico

En total se llevaron a cabo siete ensayos, de los cuales los primeros cuatro correspondieron al análisis del problema directo, mientras que los tres restantes se orientaron a la resolución del problema inverso. Tal como se detalló en la subsección 3.1.1, se empleó una única expresión para la fuerza de excitación.

A continuación, se presentan los resultados de los ensayos que aportaron evidencia concluyente para la evaluación del desempeño de los modelos PINN desarrollados.

### 4.2.1. Problema directo

La tabla 4.1 describe el conjunto de datos del ensayo a exponer, donde se indican las cantidades de puntos por cada subconjunto de puntos de colocación.

TABLA 4.1. Cantidad de puntos por cada subconjunto de puntos de colocación para el problema inverso del modelo específico.

| Subconjunto | Valor  |
|-------------|--------|
| $N_d$       | 10 000 |
| $N_{b_L}$   | 2 000  |
| $N_{b_R}$   | 2 000  |
| $N_{ci}$    | 2 000  |

Para evaluar el desempeño del modelo PINN, se generó un conjunto de 1 000 puntos aleatorios dentro del dominio de la EDP, distintos de los utilizados durante el proceso de entrenamiento.

La tabla 4.2 enlista los hiperparámetros que definieron al modelo PINN. Los valores o características expuestas se corresponden con las definiciones realizadas previamente.

TABLA 4.2. Hiperparámetros del modelo PINN que resuelve el problema directo.

| Descripción                               | Valor      |
|-------------------------------------------|------------|
| Número de capas ocultas                   | 4          |
| Número de neuronas por capa               | 20         |
| Función de activación                     | $\tanh(x)$ |
| Normalización de las variables de entrada | $[-1,1]$   |
| <i>Learning rate</i>                      | 0,1        |
| <i>epochs</i>                             | 15 000     |

La figura 4.1 muestra las curvas de convergencia de las distintas componentes de la función de pérdida, en concordancia con los subconjuntos de puntos de colocación definidos en la ecuación 3.14.

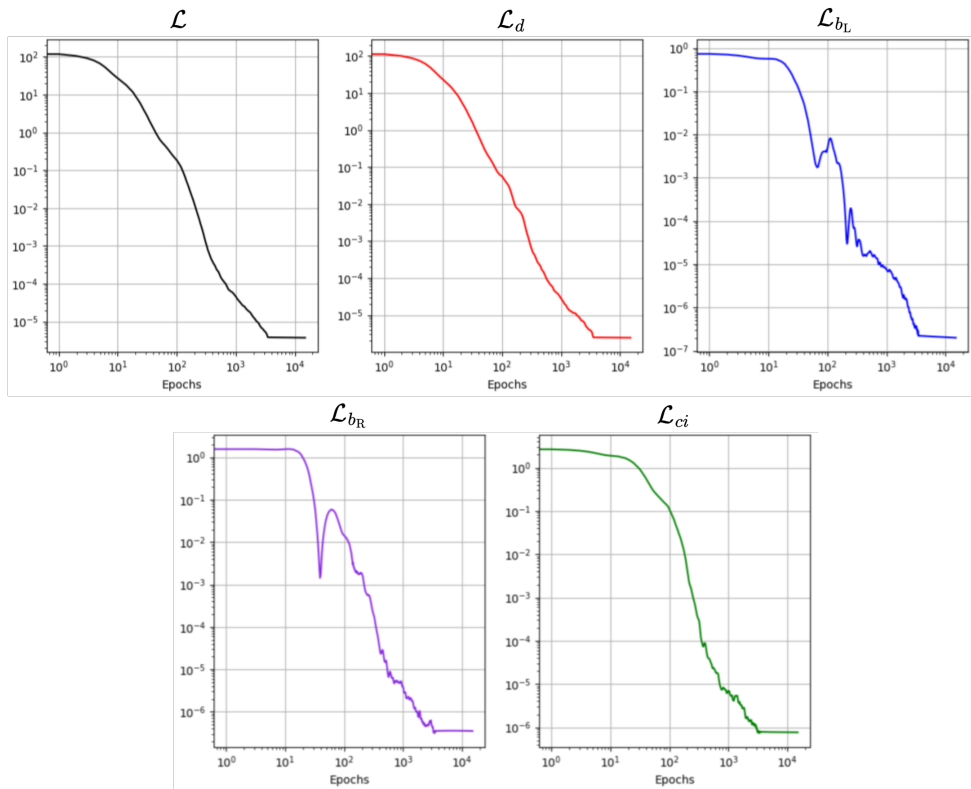


FIGURA 4.1. Convergencia de las pérdidas para la resolución del problema directo del modelo específico.

Como parte de la evaluación de los resultados, se calcularon los errores relativos  $L^2$  para las variables  $W$  y  $\Psi$ . Los valores obtenidos se visualizan en la tabla 4.3.

Por último, la figura 4.2 ilustra la capacidad de respuesta del modelo PINN entrenado. La inferencia de desplazamientos  $W$  (figura 4.2a) y de rotaciones  $\Psi$  (figura 4.2c) incluye el cálculo del error absoluto en todo el dominio, en comparación con la solución analítica dada por la ecuación 3.8.

TABLA 4.3. Valores del error relativo  $L^2$  obtenidos de la resolución del problema directo del modelo físico directo.

| Variable | Valor                |
|----------|----------------------|
| $W$      | $3,55 \cdot 10^{-4}$ |
| $\Psi$   | $3,12 \cdot 10^{-4}$ |

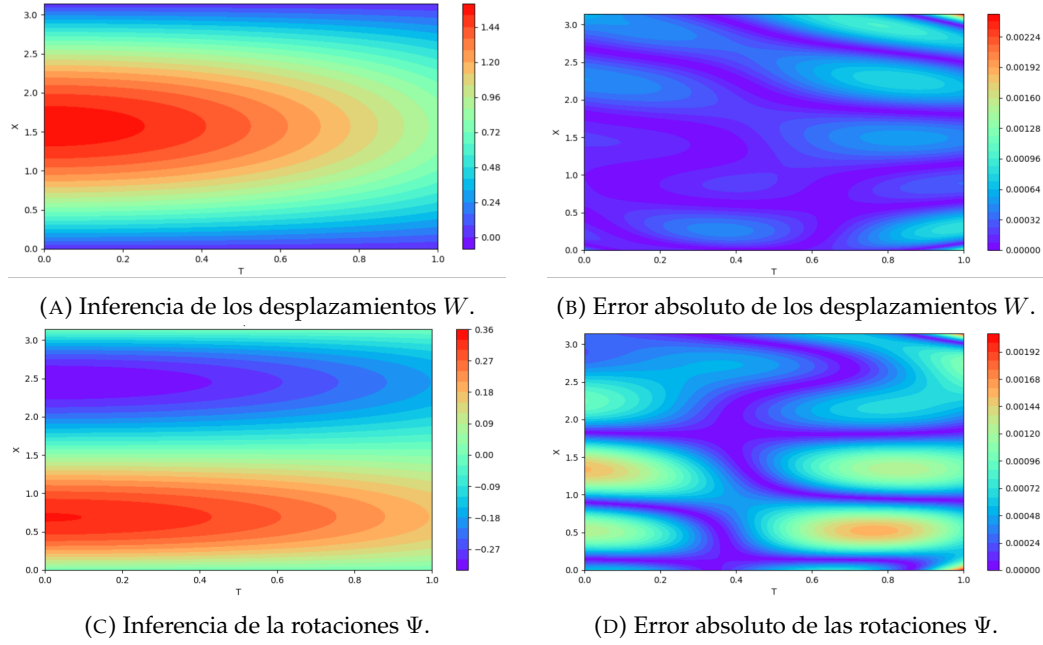


FIGURA 4.2. Inferencias y mapeo del error absoluto de los desplazamientos y rotaciones.

Los resultados obtenidos han cumplido con las expectativas proyectadas para este planteo.

#### 4.2.2. Problema inverso

Previo a describir las características y resultados de los ensayos, se rotulan los casos referidos al problema inverso analizados para facilitar su descripción:

- CI N°1: cómputo del parámetro desconocido  $\alpha$ , según la ecuación 3.6.
- CI N°2: cómputo de los parámetros desconocidos de las fuerzas de excitación  $\beta$ ,  $\gamma$  y  $\theta$ , según la ecuación 3.7.

Los casos CI N°1 y CI N°2 compartieron el mismo conjunto de datos. La tabla 4.4 describe el conjunto de datos del ensayo a exponer, en el que se indican las cantidades de puntos por cada subconjunto de puntos de colocación. En esta oportunidad se incluyó al subconjunto  $N_{data}$ , que representó a los datos rotulados.

Los datos rotulados se distribuyeron de manera equitativa en las coordenadas espaciales  $[0, 2; 0, 8; 1, 8; 2, 6; 3]$ . La figura 4.3 muestra la distribución de cada subconjunto de puntos y datos en todo el dominio espacio-temporal.

TABLA 4.4. Cantidad de puntos por cada subconjunto de puntos de colocación para el problema inverso del modelo específico.

| Subconjunto | Valor |
|-------------|-------|
| $N_d$       | 8000  |
| $N_{b_L}$   | 200   |
| $N_{b_R}$   | 200   |
| $N_{ci}$    | 200   |
| $N_{data}$  | 1 000 |

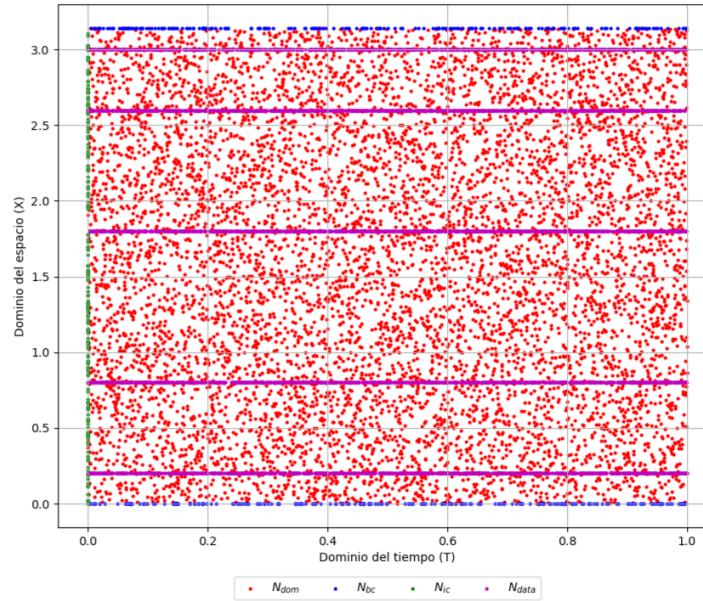


FIGURA 4.3. Puntos de colocación en el dominio espacio-temporal para la resolución del problema inverso del modelo específico.

También los modelos PINN se compartieron entre CI N°1 y CI N°2. La tabla 4.5 enlista los hiperparámetros que definieron al modelo PINN. Los valores o características expuestas se corresponden con las definiciones realizadas previamente.

TABLA 4.5. Hiperparámetros del modelo PINN que resuelve el problema inverso.

| Descripción                               | Valor CI N°1 | Valor CI N°2 |
|-------------------------------------------|--------------|--------------|
| Número de capas ocultas                   | 4            | 4            |
| Número de neuronas por capa               | 20           | 20           |
| Función de activación                     | $\tanh(x)$   | $\tanh(x)$   |
| Normalización de las variables de entrada | $[-1,1]$     | $[-1,1]$     |
| <i>Learning rate</i>                      | 0,01         | 0,001        |
| <i>epochs</i>                             | 15 000       | 15 000       |

Los resultados de la resolución del problema inverso para CI N°1 y CI N°2 se exponen en la tabla 4.6. En ambos casos, el error relativo  $L^2$  es entre una y dos órdenes de magnitud superior al equivalente calculado para la resolución del problema directo. Sin embargo, se ubicaron dentro de los valores que pueden considerarse como satisfactorios.

TABLA 4.6. Resultados para el problema inverso del modelo específico.

| Caso | $\alpha$ | $\beta$ | $\gamma$ | $\theta$ | $e_{L^2}W$           | $e_{L^2}\Psi$        |
|------|----------|---------|----------|----------|----------------------|----------------------|
| N°1  | 0,97     | -       | -        | -        | $4,45 \cdot 10^{-2}$ | $3,94 \cdot 10^{-2}$ |
| N°2  | -        | -0,0479 | 0,77     | 1,31     | $7,71 \cdot 10^{-3}$ | $3,72 \cdot 10^{-2}$ |

En lo que respecta al comportamiento de las pérdidas parciales y total, la convergencia fue similar a la observada en el problema directo. La única diferencia radica en la incorporación de pérdidas adicionales, derivadas del uso de datos rotulados.

A continuación, se presentan las inferencias y los cálculos del error absoluto correspondientes a CI N°2, ya que ofrecieron los resultados más relevantes en relación con la estimación aproximada de la fuerza de excitación. A la evaluación de los desplazamientos y rotaciones se le sumó la fuerza de excitación  $F_a^1$  (se excluye de este análisis a  $F_a^0$  porque es nula). La figura 4.4 ilustra la capacidad de respuesta del modelo PINN para resolver el problema inverso del CI N°2.

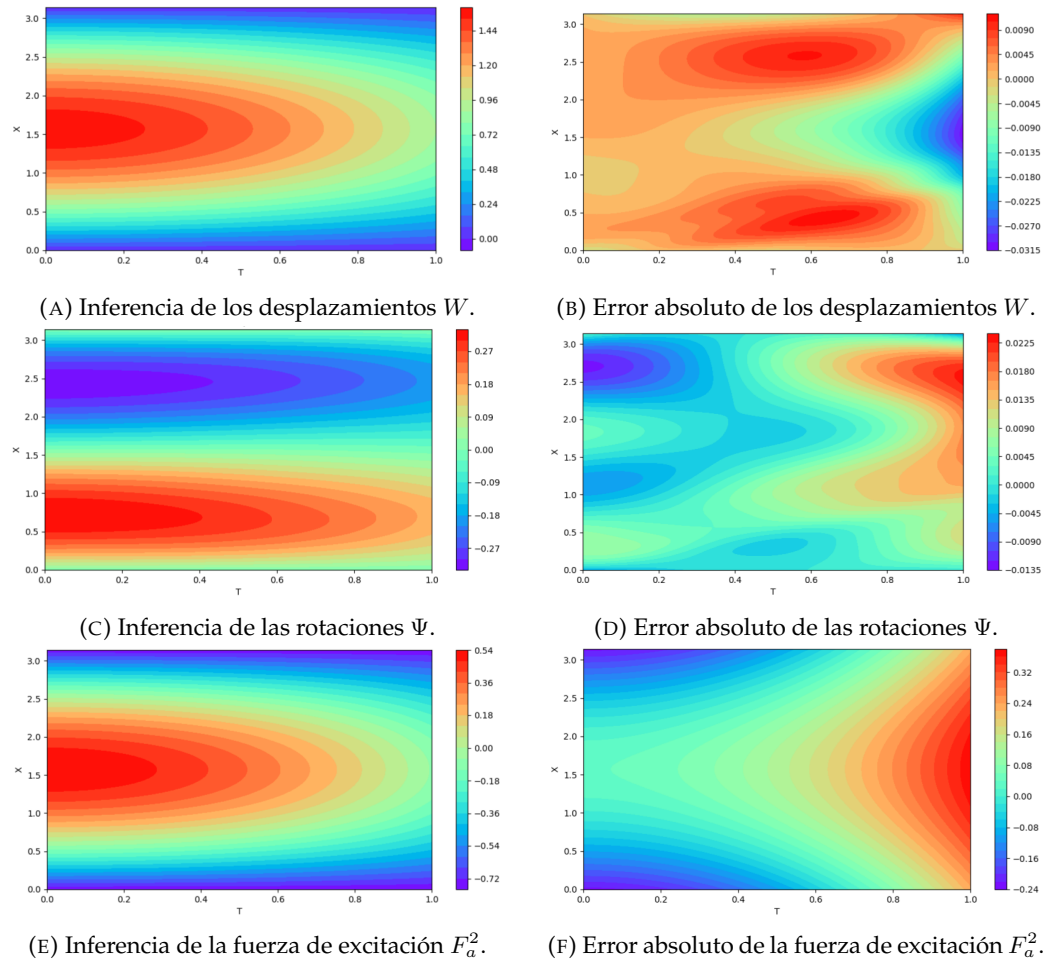


FIGURA 4.4. Inferencias y mapeo del error absoluto de los desplazamientos, rotaciones y fuerza de excitación para CI N°2.

Si bien no alcanzan la precisión lograda en el problema directo, los resultados obtenidos son considerados válidos dentro de los alcances definidos para este

trabajo. Cabe destacar que no se observaron inconvenientes en el cumplimiento de las condiciones de borde (figuras 4.4b y 4.4d). Asimismo, la aproximación de las fuerzas de excitación fue aceptable (figuras 4.4e y 4.4f). Es importante tener en cuenta que el parámetro  $\gamma$  reemplaza al término  $\cos(T)$  de la ecuación 3.5.

### 4.3. Ensayos del modelo general

El desarrollo de un modelo PINN capaz de replicar las características del modelo físico general requirió la realización de aproximadamente una veintena de ensayos. Estos incluyeron desde la corrección de errores en el código hasta la exploración de distintas alternativas para implementar la representación matemática del fenómeno físico.

Como referencia, en la tabla 4.7 se presentan las cantidades de puntos utilizadas en cada subconjunto de puntos de colocación. Aunque esta configuración no se aplicó de manera estricta en todos los casos, sirve como guía, ya que los valores empleados se mantuvieron dentro de ese rango. Asimismo, se incluye el número de puntos adicionales incorporados en el dominio con el objetivo de mejorar el desempeño del modelo, tal como se mencionó en la subsección 4.1.2.

TABLA 4.7. Cantidad de puntos de referencia por cada subconjunto de puntos de colocación para problem inverso.

| Subconjunto                         | Valor          |
|-------------------------------------|----------------|
| $N_d$                               | 15 000         |
| $N_{b_L}$                           | 1 600          |
| $N_{b_R}$                           | 1 600          |
| $N_{ci}$                            | 3 500          |
| Rango de puntos complementarios EDP | [1 200; 3 200] |

Para completar la descripción del marco general de ensayos, la tabla 4.8 presenta el rango de valores de los hiperparámetros utilizados en la evaluación de distintos modelos PINN. Los valores y características indicados se corresponden con las definiciones establecidas previamente en este trabajo.

TABLA 4.8. Rango de valores de los hiperparámetros para el modelo general.

| Descripción                               | Rango de valores |
|-------------------------------------------|------------------|
| Número de capas ocultas                   | [4; 5]           |
| Rango de neuronas por capa                | [40; 100]        |
| Función de activación                     | $\tanh(x)$       |
| Normalización de las variables de entrada | [-1,1]           |
| <i>Learning rate</i>                      | [0,1; 0,001]     |
| <i>epochs</i>                             | [10 000; 15 000] |

En cuanto a la fuente de excitación, se simuló una excitación puntual de tipo sinusoidal aplicada en el centro de la viga. La amplitud de dicha excitación se ajustó en función del análisis de los resultados obtenidos en cada prueba.

Evaluar la capacidad de un modelo PINN para admitir múltiples valores de rigidez, es decir, su aptitud para gestionar condiciones de borde variables, resulta relevante como paso hacia el desarrollo de herramientas aplicables al monitoreo de vibraciones mecánicas en cañerías, donde las circunstancias pueden variar a lo largo de toda la red. Esta variabilidad incluye tanto las condiciones iniciales como las fuentes de excitación.

En los ensayos realizados, no se logró obtener un modelo PINN que reproduzca adecuadamente el comportamiento del modelo físico planteado en la subsección 3.1.2. Sin embargo, se observó que fue capaz de responder de manera coherente a la fuerza de excitación aplicada, al menos en términos de dirección y zona de influencia, bajo las siguientes circunstancias:

- Variaciones arbitrarias en los parámetros característicos del material.
- Ajustes en las ecuaciones que definen las condiciones de borde.

Resulta relevante mencionar que los puntos mencionados carecen de una interpretación física rigurosa. Esto evidencia la necesidad de profundizar en el estudio de la representación matemática del fenómeno, así como en la estrategia de adimensionamiento de los parámetros del modelo físico, con el fin de lograr una interacción más adecuada con el modelo de deep learning.

En las conclusiones se abordará la discusión sobre las limitaciones encontradas al intentar desarrollar un modelo físico capaz de admitir múltiples configuraciones. Asimismo, se presentarán propuestas orientadas a mejorar el análisis de la respuesta dinámica y se explorará una alternativa para el estudio en el rango de alta frecuencia.





## Capítulo 5

# Conclusiones

En este capítulo se presentan las conclusiones derivadas del análisis de los resultados obtenidos en este trabajo. Se exponen las condiciones bajo las cuales los resultados fueron aceptables y las dificultades que surgieron al incrementar el grado de generalidad del problema propuesto. Asimismo, se detallan los pasos necesarios para continuar con el estudio y se plantea una propuesta de desarrollo orientada a facilitar su integración con otras tecnologías maduras.

### 5.1. Conclusiones generales

El problema abordado en este trabajo presentó dos aspectos contrapuestos. Por un lado, la representación del modelo físico se restringió a una viga unidimensional, esbelta, prismática, elástica y homogénea. Si bien esta simplificación no refleja con total fidelidad el comportamiento de un tramo de cañería, resultó adecuada para los objetivos planteados.

Por otro lado, esta elección no implicó una simplificación del fenómeno de las vibraciones mecánicas, cuya representación matemática se entrelaza con conceptos de resistencia de materiales y mecánica del sólido. Las complejidades subyacentes se hicieron evidentes en los resultados obtenidos.

En relación con el modelo físico específico, los resultados obtenidos al resolver el problema directo permitieron verificar que es posible abordar un problema de mecánica cuya representación matemática está dada por EDPs con dos variables independientes y dos dependientes, de manera directa. En cuanto al problema inverso, los resultados también fueron satisfactorios.

Es importante destacar que esto se logró utilizando un modelo PINN de baja complejidad, que requirió un conjunto de datos reducido para su entrenamiento, lo que resulta relevante. No obstante, este desempeño se alcanzó en un contexto en el que el modelo presentaba una capacidad de generalización limitada.

La situación fue diametralmente opuesta al ensayar el modelo físico general. El intento de aproximarse a una situación más cercana a la aplicación ingenieril puso de manifiesto las limitaciones actuales que presentan las PINNs para abordar determinados fenómenos físicos o para procesar distintos grados de complejidad en su representación matemática. A pesar de la gran cantidad de ensayos realizados, no fue posible encontrar un conjunto de configuraciones que ofreciera una respuesta aceptable. El modelo PINN no logró reproducir fielmente el fenómeno físico, aunque superficialmente parecía converger.

## 5.2. Próximos pasos

A continuación, se presentan los puntos que deberán recibir especial atención en futuros desarrollos, que no pudieron abordarse por exceder el alcance del presente trabajo:

- Estudio y selección de una estrategia de adimensionamiento que permita una adecuada interacción entre la representación matemática del fenómeno físico y las particularidades del modelo de red neuronal.
- Desarrollo de estrategias para trabajar con modelos oscilatorios en el ámbito de la mecánica del sólido.

Respecto al último punto, puede mencionarse el trabajo de Söyleyici y Ünver [24], quienes proponen una metodología para abordar el problema del sesgo espectral en el aprendizaje de alta frecuencia. Se trata de un algoritmo que emplea el kernel tangente de la red neuronal para ajustar los parámetros  $\lambda$  durante el entrenamiento.

Actualmente, existen programas basados en MEF con un alto grado de desarrollo, capaces de abordar este tipo de problemas con aplicaciones ingenieriles directas. Un ejemplo destacado es el software Caesar II [38], que permite modelar en tres dimensiones redes completas de cañerías, analizar vibraciones mecánicas y calcular deformaciones y tensiones. También se pueden obtener resultados similares mediante el software de código abierto Code Aster [39].

Por lo tanto, el desafío de alcanzar resultados equivalentes mediante modelos del campo de la inteligencia artificial aplicada a la ciencia es considerable. Sin embargo, estos programas tradicionales no permiten resolver el problema inverso, es decir, estimar parámetros desconocidos del modelo físico. En este aspecto, PINN y los métodos complementarios presentan un potencial significativo.

En este sentido, una línea de desarrollo propuesta consiste en centrar el estudio del fenómeno de vibraciones mecánicas exclusivamente en los soportes. Esta aproximación permitiría una simplificación del modelo físico, facilitaría su implementación y, al mismo tiempo, proporcionaría la información necesaria para optimizar las tareas que requieran el uso de programas de simulación avanzados.

Es importante destacar que la posibilidad que ofrecen los algoritmos PINN para resolver EDPs ha motivado su exploración en múltiples campos de la ciencia y la ingeniería. No obstante, este proceso ha enfrentado diversas dificultades, ya sea por las características propias del fenómeno físico que se busca representar como por los desafíos que plantea la aplicación en la que se proyecta PINN como solución. Esta situación ha dado lugar al desarrollo de numerosas técnicas complementarias que permiten superar las complicaciones de implementación. Algunos de estos métodos, que han demostrado ser efectivos en distintos casos, comienzan a considerarse como referencia [1].

## Apéndice A

# Obtención de los parámetros adimensionales

En este anexo se presenta el procedimiento utilizado para obtener los parámetros adimensionales empleados en las expresiones de las ecuaciones correspondientes a cada una de las variantes del modelo físico (ver sección 3.1).

### A.1. Modelo físico específico

El objetivo de esta sección es presentar la formulación de los parámetros adimensionales que intervienen en las ecuaciones del modelo físico específico. En este caso, debido a la forma en que están planteadas las ecuaciones de gobierno, no resulta necesario conocer el valor exacto de las constantes utilizadas en su formulación, por lo que no se detallan sus unidades.

Las constantes físicas y geométricas necesarias para determinar los parámetros adimensionales son las siguientes:

- $\rho$ : densidad del material.
- $A$ : área de la sección de la viga.
- $E$ : módulo de Young.
- $G$ : módulo de corte.
- $I$ : momento de inercia respecto al eje  $x - x$  de la sección transversal de la viga.
- $k$ : factor de corrección de corte.
- $l$ : largo de la viga.

Las variables independientes adimensionales se obtienen a partir de las siguientes expresiones

$$X = \frac{x}{l} \quad T = t \sqrt{\frac{k G}{\rho l^2}} \quad (\text{A.1})$$

donde  $x$  es la posición en el eje principal de la viga y  $t$  el tiempo.

Las variables dependientes adimensionales se definen como

$$W(X, T) = \frac{w(x(X), t(T))}{l} \quad \Psi(X, T) = \psi(x(X), t(T)) \quad (\text{A.2})$$

donde  $w$  y  $\psi$  son los desplazamientos y torsiones transversales, respectivamente.

La expresión adimensional de la fuerza externa es la siguiente

$$F_a = \frac{f_a l}{k G A} \quad (\text{A.3})$$

donde  $f_a$  es la fuerza externa transversal por unidad de longitud.

Por último, se determinan los intervalos de los dominios espacial y temporal en su forma adimensionalizada,

$$x \in [0, \pi^2] \rightarrow X \in [0, \pi] \quad (\text{A.4a})$$

$$t \in [0, t_f] \rightarrow T \in [0, 1] \quad (\text{A.4b})$$

## A.2. Modelo físico general

En esta sección se abordan dos cuestiones. Primero, se presentan las expresiones que permiten obtener los parámetros involucrados en las ecuaciones de gobierno, formuladas en su versión adimensional. Luego, se calculan los valores específicos de dichos parámetros con base en las particularidades del modelo físico general.

### A.2.1. Definiciones

Las constantes físicas y geométricas necesarias son:

- $\kappa$ : factor de corrección de corte.
- $E$ : módulo de Young  $GPa$  ( $[N/m]$ ).
- $G$ : módulo de corte  $GPa$  ( $[N/m]$ ).
- $I_0$ : momento de inercia de área respecto al eje principal de la viga  $[m^4]$ .
- $A_0$ : área de la sección de corte de la viga  $[m^2]$ .
- $L$ : largo de la viga  $[m]$ .
- $\rho$ : densidad del material  $kg/m^3$ .

A su vez, se requiere de las siguientes constantes complementarias

$$r_0 = \sqrt{\frac{I_0}{A_0}} \quad t_0 = \sqrt{\frac{\rho A_0 L^4}{E I_0}} \quad m_b = \rho A_0 L \quad (\text{A.5})$$

donde  $r_0$  es el radio de giro en  $[m]$ ,  $t_0$  el tiempo característico en  $[s]$  y  $m_b$  la masa de la viga en  $[kg]$ .

En primer lugar, se definen las constantes adimensionales

$$\gamma_{bs} = \frac{E}{\kappa G} \quad R_0 = \frac{r_0}{L} \quad (\text{A.6})$$

Las variables independientes adimensionales del modelo son

$$X = \frac{x}{L} \quad T = \frac{t}{t_0} \quad (\text{A.7})$$

donde  $x$  es la posición en el eje principal de la viga en  $[m]$  y  $t$  el tiempo en  $[s]$ .

Las variables dependientes del modelo son

$$W(X, T) = \frac{w(x(X), t(T))}{L} \quad \Psi(X, T) = \psi(x(X), t(T)) \quad (\text{A.8})$$

donde  $w$  y  $\psi$  son los desplazamientos y torsiones transversales, respectivamente.

El parámetro adimensional de la masa puntual se expresa como

$$M_1 = \frac{m_1}{m_b} \quad (\text{A.9})$$

donde  $m_1$  es el valor de la masa puntual en  $[kg]$ .

En cuanto a la representación adimensional de las elasticidades, para facilitar la nomenclatura, cualquier referencia que involucre por igual a cualquiera de ellas se la mencionará con el subíndice  $\alpha$ . Los términos adimensionales empleados se definen como

$$K_\alpha = \frac{k_\alpha L^3}{EI_0} \quad K_{t\alpha} = \frac{k_{t\alpha} L}{EI_0} \quad (\text{A.10})$$

donde  $k_\alpha$  es la elasticidad lineal en  $[N/m]$  y  $k_{t\alpha}$  es la elasticidad torcional en  $[N/m \text{ rad}]$ .

Por último, se menciona la fuerza externa distribuida a lo largo de la viga de forma adimensionalizada

$$F_a(X, T) = \frac{f_a(x(X), t(T)) L^3}{EI_0} \quad (\text{A.11})$$

donde  $f_a$  es la fuerza externa transversal por unidad de longitud expresada en  $[N/m]$ .

### A.2.2. Cálculo de valores

Las propiedades de la viga inicialmente fueron tomadas del trabajo de Chung y coautores [40]. No obstante, se modificó la longitud de la viga con el objetivo de acentuar el comportamiento característico del modelo PINN. Por otra parte, los parámetros adimensionales relacionados a rigideces, fuerza de excitación y elementos puntuales se calculan de acuerdo a la configuración particular del ensayo a realizar. Esta condición requirió considerar el valor de las propiedades geométricas y del material de la viga, según lo reportado por Grant en su estudio sobre el efecto de las deformaciones en los modos de vibración de una viga [41].

Valores de las propiedades geométricas y del material de la viga que son constantes

$$\begin{aligned} L &= 1 \text{ m}^2 & A_0 &= 2,54 \cdot 10^{-5} \text{ m}^2 & I_0 &= 3,12 \cdot 10^{-7} \text{ m}^4 \\ \rho &= 7680 \frac{\text{kg}}{\text{m}^3} & E &= 207 \text{ GPa} & \frac{E}{G} &= 2,667 & \kappa &= \frac{\pi^2}{12} = 0,822 \end{aligned}$$

Valores a tener en cuenta para la modificación de parámetros adimensionales

$$R_0^2 = 1,23 \cdot 10^{-2} \quad \gamma_{bs} = \frac{E}{\kappa G} = 3,243$$

Finalmente, se definen los valores de las posiciones de los elementos puntuales

$$X_{m1} = 0,25 \quad X_1 = 0,50 \quad X_{t1} = 0,75$$

Respecto a las cotas del dominio de las ecuaciones de gobierno, el dominio espacial adimensionalizado presenta el siguiente intervalo de valores

$$x \in [0; 1] \, m \quad \rightarrow \quad X \in [0; 1]$$

Resta conocer el valor del tiempo característico de la viga para definir el dominio temporal adimensionalizado. De acuerdo a la ecuación [A.5](#) se tiene que

$$t_f \in [5; 10] \, s \quad \rightarrow \quad T_f \in [2\,876,97; 5\,753,94]$$

# Bibliografía

- [1] Y. Wang et al. «Artificial intelligence for partial differential equations in computational mechanics: A review». En: *arXiv preprint 2410.19843v2* (2024).
- [2] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016.
- [3] A. Krizhevsky et al. «ImageNet classification with deep convolutional neural networks». En: *Advances in neural information processing systems 25* (2012), 1097–1105.
- [4] A. Graves et al. «Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks». En: *ICML '06: Proceedings of the 23rd international conference on Machine learning* (2006), págs. 369–376.
- [5] T. Brown et al. «Language models are few-shot learners». En: *Advances in neural information processing systems 33* (2020), 1877–1901.
- [6] D. Silver y A. Huang et al. «Mastering the game of go with deep neural networks and tree search». En: *Nature 529* (7587) (2016), págs. 484–489.
- [7] O. Vinyals et al. «Grandmaster level in starcraft ii using multi-agent reinforcement learning». En: *Nature 575* (7782) (2019), págs. 350–354.
- [8] A. W. Senior et al. «Improved protein structure prediction using potentials from deep learning». En: *Nature 577* (7792) (2020), págs. 706–710.
- [9] X. Zhang et al. «intelligence for science in quantum, atomistic, and continuum systems». En: *arXiv preprint 2307.08423* (2023), págs. 706–710.
- [10] M. Raissi, P. Perdikaris y G. E. Karniadakis. «Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations». En: *Journal of Computational Physics, vol 378* (2019), págs. 686–707.
- [11] G.E. Karniadakis et al. «Physics-informed machine learning». En: *Nature Reviews Physics 3.6* (2021), págs. 442–440.
- [12] Jorge Nocedal y Stephen J. Wright. *Numerical Optimization*. Second edition. Springer Series in Operations Research and Financial Engineering. Springer, 2006.
- [13] O. C. Zienkiewicz y R. L. Taylor. *The Finite Element Method: Its Basis and Fundamentals*. Butterworth-Heinemann, 2013.
- [14] F. Lauer et al. «Incorporating prior knowledge in support vector machines for classification: A review». En: *Neurocomputing 71* (7–9) (2008), págs. 1578–1594.
- [15] H. Lee e I. S. Kang. «Neural algorithm for solving differential equations». En: *Journal of Computational Physics 91* (1) (1990), págs. 110–131.
- [16] M. Dissanayake y N. Phan-Thien. «Neural-network-based approximations for solving partial differential equations». En: *Communications in Numerical Methods in Engineering 10* (3) (1994), págs. 195–201.
- [17] I. E. Lagaris et al. «Artificial neural networks for solving ordinary and partial differential equations». En: *IEEE transactions on neural networks 9* (5) (1998), págs. 987–1000.
- [18] *PyTorch*. En: *Wikipedia*. 2025. URL: <https://en.wikipedia.org/w/index.php?title=PyTorch&oldid=1322419216>.

- [19] TensorFlow. En: Wikipedia. 2025. URL: <https://en.wikipedia.org/w/index.php?title=TensorFlow&oldid=1321137906>.
- [20] Zhi-Qin John Xu et al. «Training behavior of deep neural network in frequency domain». En: *ICONIP 2019, Lecture Notes in Computer Science()*, vol. 11953 (2019).
- [21] C.A. Yan et al. «A framework based on physics-informed neural networks and extreme learning for the analysis of composite structures». En: *Computers & Structures*, vol. 265 (2022).
- [22] A. Fallah y M. M. Aghdam. «Physics-informed neural network for bending and free vibration analysis of three-dimensional functionally graded porous beam resting on elastic foundation». En: *Engineering with Computers* 40 (2023), págs. 437-454.
- [23] T. Kapoor et al. «Physics-Informed Neural Networks for Solving Forward and Inverse Problems in Complex Beam Systems». En: *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, issue 5 (2024).
- [24] C. Söyleyici y H.O. Ünver. «A Physics-Informed Deep Neural Network based beam vibration framework for simulation and parameter identification». En: *Engineering Applications of Artificial Intelligence*, vol. 141 (2025).
- [25] Vladimir Stojanović y Predrag Kozić. *Vibrations and Stability of Complex Beam Systems*. Springer Tracts in Mechanical Engineering. Springer International Publishing, 2015. DOI: [10.1007/978-3-319-13767-4](https://doi.org/10.1007/978-3-319-13767-4).
- [26] G. Bai et al. «Physics Informed Neural Networks (PINNs) for approximating nonlinear dispersive PDEs». En: *arXiv preprint 2408.07027* (2022).
- [27] S. P. Timoshenko y J. M. Gere. *Mechanics of Materials*. New York: Van Nostrand Reinhold Company, 1972.
- [28] Notación para la diferenciación. En: Wikipedia. 2025. URL: [https://es.wikipedia.org/w/index.php?title=Notaci%C3%B3n\\_para\\_la\\_diferenciaci%C3%B3n&oldid=169442918](https://es.wikipedia.org/w/index.php?title=Notaci%C3%B3n_para_la_diferenciaci%C3%B3n&oldid=169442918).
- [29] Vishwajeet. *Why Python Is the Leader in Artificial Intelligence and Data Science*. Medium. URL: <https://vishwajeetsinghrama8.medium.com/why-python-is-the-leader-in-artificial-intelligence-and-data-science-57ef9f8dfc88>.
- [30] NumPy. En: Wikipedia. 2025. URL: <https://en.wikipedia.org/w/index.php?title=NumPy&oldid=1320348672>.
- [31] CUDA. En: Wikipedia. 2025. URL: <https://en.wikipedia.org/w/index.php?title=CUDA&oldid=1323192092>.
- [32] Notion (Productivity Software). En: Wikipedia. 2025. URL: [https://en.wikipedia.org/w/index.php?title=Notion\\_\(productivity\\_software\)&oldid=1314437502](https://en.wikipedia.org/w/index.php?title=Notion_(productivity_software)&oldid=1314437502).
- [33] Carlos G. Massobrio. *Estudio de vibraciones mecánicas en vigas con PINN*. 2025. URL: [https://github.com/cgmassobrio/vibraciones-mecanicas\\_CEIA-FIUBA](https://github.com/cgmassobrio/vibraciones-mecanicas_CEIA-FIUBA).
- [34] Google Colab. En: Wikipedia. 2025. URL: [https://en.wikipedia.org/w/index.php?title=Google\\_Colab&oldid=1321087407](https://en.wikipedia.org/w/index.php?title=Google_Colab&oldid=1321087407).
- [35] Debian. En: Wikipedia. 2025. URL: <https://en.wikipedia.org/w/index.php?title=Debian&oldid=1322943175>.
- [36] Visual Studio Code. En: Wikipedia. 2025. URL: [https://en.wikipedia.org/w/index.php?title=Visual\\_Studio\\_Code&oldid=1323212280](https://en.wikipedia.org/w/index.php?title=Visual_Studio_Code&oldid=1323212280).
- [37] Conda (Package Manager). En: Wikipedia. 2025. URL: [https://en.wikipedia.org/w/index.php?title=Conda\\_\(package\\_manager\)&oldid=1316311385](https://en.wikipedia.org/w/index.php?title=Conda_(package_manager)&oldid=1316311385).
- [38] CAESAR II: Piping Stress Analysis Overview. 2024. URL: <https://www.cortexsoftware.com.au/blog/understanding-caesar-ii-the-industry-leading-piping-flexibility-and-stress-analysis-software>.



- [39] *Salome (Software)*. En: *Wikipedia*. 2025. URL: [https://en.wikipedia.org/w/index.php?title=Salome\\_\(software\)&oldid=1310738178](https://en.wikipedia.org/w/index.php?title=Salome_(software)&oldid=1310738178).
- [40] J. H. Chung, W. H. Joo y K. C. Kim. «Vibration And Dynamic Sensitivity Analysis Of A Timoshenko Beam-Column With Elastically Restrained Ends And Intermediate Constraints». En: *Journal of Sound and Vibration* 167.2 (1993), págs. 209-221. DOI: [10.1006/jsvi.1993.1331](https://doi.org/10.1006/jsvi.1993.1331).
- [41] D. A. Grant. «The Effect of Rotary Inertia and Shear Deformation on the Frequency and Normal Mode Equations of Uniform Beams Carrying a Concentrated Mass». En: *Journal of Sound and Vibration* 57.3 (1978), págs. 357-365. DOI: [10.1016/0022-460X\(78\)90316-4](https://doi.org/10.1016/0022-460X(78)90316-4).